

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN TRÍ TUỆ NHÂN TẠO



HUỖNH THANH NHẢN – 23110280
TRƯỜNG TẢN SANG – 23110300
HUỖNH NGỌC THẮNG – 23110327

Đề Tài :

ỨNG DỤNG HỌC TĂNG CƯỜNG TRONG MÔI TRƯỜNG
MULTI – FLOOR MAZE
ĐỒ ÁN MÔN HỌC TĂNG CƯỜNG

GIÁO VIÊN HƯỚNG DẪN
THS. LÊ MINH TÂN

KHÓA 2023 – 2027

LỜI CẢM ƠN

Lời đầu tiên, nhóm xin được gửi lời cảm ơn chân thành nhất đến ThS. Lê Minh Tân. Qua những bài học, lời khuyên và sự đóng góp của thầy đã giúp nhóm hiểu hơn về môn học Học tăng cường cũng như trong quá trình hoàn thành đồ án cuối kỳ.

Do thời gian đợt học lần này đã ngắn đi nên cùng với những kiến thức và bài học tiếp thu được, nhóm đã cố gắng thực hiện đề tài hết mình, cùng nhau thực hiện một cách tốt nhất với khả năng. Chính vì thế trong quá trình đôi khi sẽ có những sai sót không mong muốn, từ đó nhóm hi vọng sẽ nhận được sự góp ý từ thầy để nhóm có thể sửa chữa và rút ra bài học hướng đến mục tiêu là hoàn thiện các đồ án sau này.

Một lần nữa nhóm xin chân thành cảm ơn thầy. Kính chúc thầy thật nhiều sức khỏe, niềm vui trong sự nghiệp trồng người. Nhóm chúng em xin cảm ơn thầy.

TP. Hồ Chí Minh, ngày 24 tháng 03 năm 2026

Nhóm sinh viên thực hiện

Huỳnh Thanh Nhân

Trương Tấn Sang

Huỳnh Ngọc Thắng

MỤC LỤC

LỜI CẢM ƠN.....	
NỘI DUNG.....	
CHƯƠNG 1: LÝ THUYẾT LIÊN QUAN.....	1
1.1. Tổng quan về học tăng cường (Reinforcement learning - RL).....	1
1.1.1. Agent và Environment.....	1
1.1.2 Bài toán tối ưu tuần tự và mục tiêu tối đa hóa tổng reward.....	2
1.2. Mô hình Markov Decision Process (MDP).....	2
1.2.1. Thành phần MDP.....	2
1.2.2. Chính sách (policy) và giá trị kỳ vọng (Value function).....	3
1.2.3. Hàm giá trị theo hành động và phương trình Bellman	4
1.2.4. Reward function và reward shaping	4
1.3. Chiến lược khám phá - khai thác trong RL.....	5
1.3.1. Khám phá/khai thác (Exploration vs Exploitation)	5
1.3.2. Chính sách ϵ -greedy.....	6
1.3.3. Lịch giảm ϵ (epsilon decay).....	6
1.4. Thuật toán Monte Carlo Control (First-Visit MC)	7
1.4.1. Return (G_t) và cập nhật theo episode	7
1.4.2. Cập nhật	7
1.5. Thuật toán Q-Learning.....	8
1.5.1. Cập nhật theo TD (Temporal-Difference)	8
1.5.2. Bootstrap và công thức TD target.....	9
1.5.3. Ưu/nhược điểm so với Monte Carlo	10
1.6. Double Deep Q-Network (Double DQN)	10
1.6.1. Xấp xỉ Q-function bằng Neural Network	10
1.6.2. Chu trình huấn luyện Double DQN	10

1.6.3. Experience Replay Buffer.....	11
1.6.4 Target Network (Soft update).....	11
1.6.5 Hàm mất mát (Huber / Smooth L1) và cập nhật gradient	12
1.6.6 Action Masking	12
CHƯƠNG 2: MÔ TẢ HỆ THỐNG.....	13
2.1. Thiết kế bài toán.....	13
2.1.1. Mô tả môi trường	13
2.1.2. Cấu hình môi trường.....	15
2.1.2.1. Trạng thái đầy đủ.....	15
2.1.2.2. Trạng thái quan sát được	16
2.1.3. Không gian hành động (Action Space).....	17
2.1.4. Hệ thống phần thưởng (Reward Function).....	18
2.1.5. Điều kiện kết thúc (Terminal Conditions).....	20
2.2. Cài đặt và triển khai	20
2.2.1. Sơ đồ khối hệ thống và kiến trúc mã nguồn	20
2.2.2. Monte Carlo (cài đặt tabular).....	21
2.2.3. Q-learning (cài đặt tabular).....	23
2.2.4. Kiến trúc mạng Double DQN	24
2.2.5. Cấu hình tham số huấn luyện.....	26
2.3. Kết quả thực nghiệm và đánh giá.....	27
2.3.1. Thiết lập thực nghiệm và tiêu chí đánh giá.....	27
2.3.2. Biểu đồ hội tụ.....	28
2.3.3. So sánh Q-learning, Monte Carlo	28
2.3.3.1. Phase 1	28
2.3.3.2. Phase 2.....	31

2.3.3.3. Phase 3	34
2.3.4. Đánh giá Double DQN	37
2.3.4.1. Phase 1	37
2.3.4.2. Phase 2	38
2.3.4.3. Phase 3	40
2.3.4.4. Phase 4	41
Chương 3: Kết luận	44
3.1. Kết quả đạt được	44
3.2. Hạn chế và hướng phát triển	45

NỘI DUNG

CHƯƠNG 1: LÝ THUYẾT LIÊN QUAN

1.1. Tổng quan về học tăng cường (Reinforcement learning - RL)

Học tăng cường là một nhánh của học máy (machine learning) mà trong đó một tác tử (agent) tương tác với môi trường (environment) nhằm học cách hành động để tối đa hóa một tín hiệu phần thưởng tích lũy theo thời gian. Không giống với học có giám sát (supervised learning), RL không yêu cầu nhãn đầu ra được cung cấp sẵn, thay vào đó là tác tử phải tự khám phá chiến lược hành động tốt nhất thông qua quá trình thử và sai (trial and error) trong môi trường.

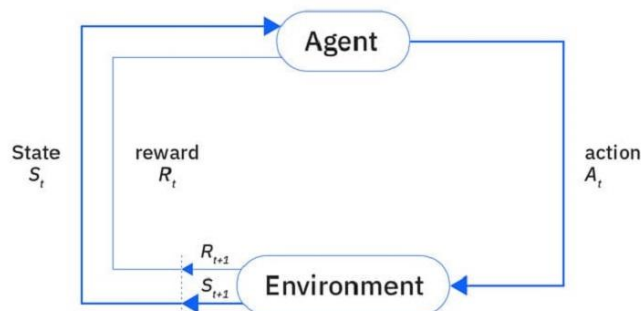
1.1.1. Agent và Environment

Trong học tăng cường, hai thực thể cơ bản nhất là *tác tử (agent)* và *môi trường (environment)*. Trong đó:

Tác tử (Agent): Là thực thể ra quyết định. Nó chịu trách nhiệm quan sát trạng thái của môi trường và đưa ra hành động tương ứng dựa trên chiến lược (policy) hiện tại của mình.

Môi trường (Environment): bao gồm tất cả những gì tác tử tương tác nhưng không trực tiếp kiểm soát. Môi trường nhận hành động của tác tử, chuyển đổi sang trạng thái tiếp theo và phát ra tín hiệu phần thưởng (reward).

Sự tương tác giữa hai thực thể này tạo thành vòng lặp học liên tục, được mô tả qua sơ đồ tương tác cơ bản sau:



Hình 1: Sự tương tác giữa agent và môi trường

Cụ thể vòng lặp tương tác: Tại mỗi bước thời gian t , agent quan sát trạng thái hiện tại S_t từ môi trường, thực hiện hành động A_t . Môi trường tiếp nhận hành động này, biến đổi sang trạng thái tiếp theo S_{t+1} và cung cấp cho agent một phần thưởng R_{t+1} .

Quá trình này lặp lại liên tục cho đến khi Agent đạt được mục tiêu (Terminal State) hoặc hết số bước cho phép.

1.1.2 Bài toán tối ưu tuần tự và mục tiêu tối đa hóa tổng reward

Học tăng cường được xây dựng trên nền tảng bài toán ra quyết định tuần tự (sequential decision-making problem). Tại mỗi bước thời gian t , tác tử quan sát trạng thái $S_t \in \mathcal{S}$, thực hiện hành động $A_t \in \mathcal{A}$, và nhận phần thưởng tức thời $R_{t+1} \in \mathbb{R}$ từ môi trường, đồng thời môi trường chuyển sang trạng thái mới S_{t+1} . Quá trình này tạo ra một quỹ đạo (trajectory): $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots)$.

Mục tiêu của tác tử không phải là tối đa hóa phần thưởng tức thời tại từng bước, mà là tối đa hóa tổng phần thưởng tích lũy có chiết khấu (discounted cumulative return) theo toàn bộ quỹ đạo.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

trong đó $\gamma \in [0, 1)$ là *hệ số chiết khấu (discount factor)*, đảm bảo tổng phần thưởng hội tụ trong các bài toán vô hạn. Trong một số bài toán hữu hạn, có thể sử dụng $\gamma = 1$. Hệ số này đóng vai trò điều chỉnh tầm nhìn dài hạn của tác tử:

- $\gamma \rightarrow 0$: tác tử thiên cận, chú trọng phần thưởng tức thời.
- $\gamma \rightarrow 1$: tác tử chiến lược.

$0 < \gamma < 1$: đảm bảo chuỗi vô hạn hội tụ, tránh tổng phần thưởng tiến đến vô cực trong môi trường không kết thúc.

1.2. Mô hình Markov Decision Process (MDP)

Markov Decision Process (MDP) là khung toán học chính thức được sử dụng phổ biến nhất để mô hình hóa các bài toán học tăng cường. MDP cung cấp ngôn ngữ toán học chặt chẽ để mô tả môi trường có tính chất Markov, tức là trạng thái tương lai chỉ phụ thuộc vào trạng thái và hành động hiện tại, không phụ thuộc vào lịch sử trước đó.

1.2.1. Thành phần MDP

Một MDP được định nghĩa hình thức bởi bộ năm thành phần: (S, A, P, R, γ) .

Ký hiệu	Thành phần	Ý nghĩa
S	Không gian trạng thái	Tập hợp tất cả các trạng thái có thể của môi trường. Có thể rời rạc (discrete) hoặc liên tục (continuous). Tại mỗi

		thời điểm t , tác tử quan sát trạng thái S_t , được xem như một biểu diễn đầy đủ thông tin cần thiết về môi trường để đưa ra quyết định hành động.
A	Không gian hành động	Tập hợp tất cả các hành động mà agent có thể thực hiện tại mỗi trạng thái. Tương tự, có thể rời rạc hoặc liên tục.
P	Hàm chuyển trạng thái. Ký hiệu là $P(s' s, a)$	Xác suất chuyển từ trạng thái s sang trạng thái s' khi thực hiện hành động a . Đặc trưng cho tính ngẫu nhiên của môi trường.
R	Hàm phần thưởng. Ký hiệu là $R(s, a, s')$	Phần thưởng tức thời mà agent nhận được khi chuyển từ trạng thái s sang s' qua hành động a .
γ	Hệ số chiết khấu	Tham số $\gamma \in [0,1)$ điều chỉnh mức độ ưu tiên phần thưởng tương lai so với hiện tại. Trong các bài toán hữu hạn, $\gamma = 1$ vẫn có thể được sử dụng.

Bảng 1: Mô tả thành phần MDP

1.2.2. Chính sách (policy) và giá trị kỳ vọng (Value function)

Chính sách (Policy - π): Chính sách π là ánh xạ từ không gian trạng thái sang không gian hành động, xác định cách thức agent chọn hành động tại mỗi trạng thái.

Chính sách có thể là:

- Chính sách tất định (Deterministic policy): $a = \pi(s)$, tại mỗi trạng thái s , Agent luôn chọn chính xác một hành động a cố định.
- Chính sách ngẫu nhiên (Stochastic policy): $\pi(a|s) = P(A_t = a|S_t = s)$, xác định phân phối xác suất để chọn hành động a khi đang ở trạng thái s .

Chính sách ngẫu nhiên thường được ưa chuộng hơn trong thực tế vì nó hỗ trợ khám phá (exploration) và có lợi trong các môi trường đối kháng (adversarial environments).

Hàm giá trị trạng thái $V^\pi(s)$ đo lường giá trị kỳ vọng của tổng phần thưởng có chiết khấu khi bắt đầu từ trạng thái s và tuân theo chính sách π .

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

Hàm này cung cấp đánh giá về *mức độ tốt* của việc ở trạng thái s dưới chính sách π . Trạng thái có $V^\pi(s)$ cao tương ứng với khả năng nhận được nhiều phần thưởng hơn trong tương lai.

1.2.3. Hàm giá trị theo hành động và phương trình Bellman

Hàm giá trị hành động (Action-Value Function / Q-function): Hàm $Q^\pi(s, a)$ đánh giá giá trị kỳ vọng của tổng phần thưởng nhận được nếu Agent bắt đầu tại trạng thái s , thực hiện một hành động cụ thể a , và sau đó tuân thủ nghiêm ngặt theo chính sách π cho các bước tiếp theo.

$$Q^\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$$

Đối với hàm giá trị hành động tối ưu Q^* , tương ứng với chính sách tối ưu thì phương trình Bellman tối ưu được phát biểu như sau: Giá trị của việc thực hiện hành động a tại trạng thái s bằng tổng của phần thưởng kỳ vọng ngay lập tức và giá trị chiết khấu lớn nhất có thể đạt được ở bước tiếp theo.

$$Q^*(s, a) = R(s, a) + \gamma \max_{a'} Q^*(s', a')$$

Trong đó $\max_{a'} Q^*(s', a')$ là giá trị Q lớn nhất trong tất cả các hành động a' có thể thực hiện tại trạng thái tiếp theo s' . Phương trình này cung cấp cơ sở để tác tử liên tục cập nhật và cải thiện sự hiểu biết của mình về môi trường.

1.2.4. Reward function và reward shaping

Thiết kế tín hiệu phần thưởng (Reward function) là bước quan trọng quyết định sự thành bại của một mô hình RL, bởi tác tử sẽ làm mọi cách để tối đa hóa điểm số này.

Tín hiệu phần thưởng thưa thớt (Sparse Reward): Trong nhiều bài toán thực tế, tác tử chỉ nhận được phần thưởng khi đã hoàn thành toàn bộ mục tiêu (ví dụ: +1 khi thắng ván cờ). Điều này khiến tác tử rất khó học trong giai đoạn đầu vì xác suất ngẫu nhiên đi đến đích là cực kỳ nhỏ, dẫn đến việc thiếu thông tin phản hồi.

Định hình phần thưởng (Reward Shaping): Để khắc phục vấn đề tín hiệu thưa thớt, kỹ thuật Reward Shaping được áp dụng nhằm cung cấp các phần thưởng trung gian (dense rewards) giúp hướng dẫn cho tác tử. Kỹ thuật này bổ sung các hàm đánh giá logic để khuyến khích hoặc trừng phạt các hành vi cụ thể mà không làm thay đổi mục tiêu tối thượng.

Ví dụ trong môi trường mê cung:

- Hình phạt thời gian (Time penalty): Trừ điểm cho mỗi bước di chuyển, ép phải tìm đường ngắn nhất.
- Khuyến khích khám phá: Thưởng điểm nhỏ khi mở khóa được cửa,...
- Trừng phạt hành vi vô ích: Trừ điểm nặng khi đâm vào tường,...

Tuy nhiên, Reward Shaping cần được thiết kế vô cùng cẩn thận để tránh hiện tượng "Reward Hacking", khi tác tử tìm ra cách thu thập phần thưởng trung gian một cách lặp đi lặp lại mà không thực sự hoàn thành mục tiêu cuối cùng.

1.3. Chiến lược khám phá - khai thác trong RL

Một trong những thách thức cơ bản và đặc trưng nhất của học tăng cường là mâu thuẫn giữa khám phá và khai thác (exploration–exploitation dilemma). Đây là bài toán ra quyết định mà bất kỳ agent nào cũng phải đối mặt: liệu agent nên *khai thác* những hành động đã biết là tốt, hay nên *khám phá* các hành động mới để có thể tìm ra chiến lược tốt hơn? Không có lời giải đúng tuyệt đối – sự cân bằng hợp lý giữa hai chiến lược này là yếu tố quyết định chất lượng và tốc độ học của agent.

1.3.1. Khám phá/khai thác (Exploration vs Exploitation)

Khai thác (Exploitation): Là quá trình tác tử đưa ra quyết định dựa hoàn toàn vào những thông tin đã học được tính đến thời điểm hiện tại. Tức là, tác tử sẽ chọn hành động mang lại giá trị kỳ vọng cao nhất (hành động tham lam) để tối đa hóa phần thưởng tức thời. Tuy nhiên, nếu chỉ tập trung vào khai thác, tác tử dễ bị mắc kẹt ở các giải pháp tối ưu cục bộ (local optimum).

Khám phá (Exploration): Là quá trình tác tử lựa chọn các hành động ngẫu nhiên hoặc chưa từng thử qua thay vì hành động tốt nhất hiện tại. Mục đích của khám phá là thu thập thêm thông tin về môi trường, từ đó có cơ hội phát hiện ra những chiến lược tốt hơn trong dài hạn.

Ví dụ minh họa: Trong bài toán tìm đường trong mê cung đa tầng, nếu tác tử chỉ khai thác, nó có thể luôn đi theo một lộ trình quen thuộc dù lộ trình này rất dài. Ngược lại, nếu tác tử chịu khám phá, nó có thể đi nhầm vào ngõ cụt vài lần, nhưng cũng có thể tình cờ phát hiện ra một lối tắt, hoặc nhặt được một vật phẩm (như búa đập tường) giúp rút ngắn đáng kể khoảng cách đến đích.

1.3.2. Chính sách ϵ -greedy

Để giải quyết bài toán đánh đổi nêu trên, chính sách ϵ -greedy (Epsilon-greedy) được áp dụng như một trong những phương pháp đơn giản nhưng hiệu quả nhất.

Thuật toán sử dụng một tham số ϵ (Epsilon) có giá trị nằm trong khoảng $[0, 1]$ để làm ngưỡng xác suất quyết định hành vi của tác tử tại mỗi bước đi:

- Với xác suất $(1 - \epsilon)$: tác tử thực hiện hành động khai thác. Nó duyệt qua Q-Table và chọn hành động có giá trị Q lớn nhất tại trạng thái hiện tại ($a = \operatorname{argmax} Q(s, a)$).
- Với xác suất ϵ : Agent thực hiện hành động Khám phá. Nó bỏ qua Q-Table và chọn hoàn toàn ngẫu nhiên một hành động bất kỳ trong không gian hành động hợp lệ.

Nếu ϵ được đặt ở mức quá cao (ví dụ: 0.9), Agent sẽ gần như di chuyển ngẫu nhiên, khó hội tụ về lộ trình tối ưu. Ngược lại, nếu ϵ quá thấp (ví dụ: 0.01), Agent sẽ hội tụ rất nhanh nhưng có nguy cơ đi theo lộ trình rất kém hiệu quả.

1.3.3. Lịch giảm ϵ (epsilon decay)

Trong thực tế, việc duy trì ϵ cố định suốt quá trình học là không tối ưu: giai đoạn đầu cần khám phá nhiều (ϵ lớn) để xây dựng kiến thức, nhưng giai đoạn sau cần khai thác nhiều hơn (ϵ nhỏ) để tận dụng kiến thức đã học. Lịch giảm ϵ (epsilon decay schedule) là cơ chế giảm dần ϵ theo thời gian. Có hai phương pháp suy giảm phổ biến:

Suy giảm tuyến tính (Linear Decay): Giá trị ϵ sẽ giảm đi một lượng hằng số sau mỗi tập huấn luyện (episode) cho đến khi đạt mức tối thiểu. Công thức: $\epsilon_t = \epsilon_{t-1} - \text{decay}_{rate}$.

Suy giảm theo hàm mũ (Exponential Decay): Đây là phương pháp phổ biến hơn trong các môi trường phức tạp như Multi-Floor Maze. Ở giai đoạn đầu, ϵ giảm nhanh, nhưng ở giai đoạn sau, tốc độ giảm chậm lại, giúp duy trì một tỷ lệ khám phá nhỏ lẻ để đề phòng môi trường có sự thay đổi. Công thức: $\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \times \exp(-\lambda \times t)$.

Trong đó:

- ϵ_t : Tỷ lệ khám phá ở tập huấn luyện thứ t .
- $\epsilon_{\max}$: Tỷ lệ khám phá khởi tạo.
- $\epsilon_{\min}$: Tỷ lệ khám phá tối thiểu.

- λ (Lambda): Tốc độ suy giảm (Decay rate).
- t : Chỉ số của tập huấn luyện hiện tại (Episode).

Bằng cách thiết lập lịch giảm ϵ hợp lý, mô hình RL sẽ chuyển đổi một cách mượt mà từ việc học hỏi môi trường (thử và sai) sang việc áp dụng triệt để những gì đã học để tìm ra đường đi tối ưu nhất đến đích.

1.4. Thuật toán Monte Carlo Control (First-Visit MC)

Monte Carlo (MC) là một nhóm phương pháp học tăng cường dựa trên việc lấy mẫu (sampling) từ các episode hoàn chỉnh để ước lượng giá trị. Các phương pháp này không yêu cầu mô hình động học của môi trường (model-free), mà học trực tiếp từ trải nghiệm thu thập được thông qua tương tác với môi trường. Điểm đặc trưng của Monte Carlo là việc cập nhật giá trị chỉ được thực hiện sau khi một episode kết thúc, dựa trên tổng phần thưởng tích lũy quan sát được. Điểm khác biệt cốt lõi so với các phương pháp lập trình động là MC chỉ sử dụng dữ liệu thực sự tương tác được.

1.4.1. Return (G_t) và cập nhật theo episode

Trong MC, toàn bộ quá trình học được tổ chức theo đơn vị episode – một chuỗi tương tác hoàn chỉnh từ trạng thái khởi đầu đến trạng thái kết thúc. Một episode có dạng:

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{t-1}, A_{t-1}, R_t, S_t)$$

Sau khi episode kết thúc, tổng phần thưởng có chiết khấu (discounted return) từ bước thời gian t được tính lùi (backward) theo công thức đệ quy

$$G_t = R_{t+1} + \gamma \cdot G_{t+1}$$

Sau khi có G_t , ước lượng hàm Q được cập nhật theo nguyên tắc trung bình mẫu:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G_t - Q(s, a)]$$

trong đó $\alpha \in (0, 1]$ là tốc độ học (learning rate).

1.4.2. Cập nhật

Trong một episode, một cặp trạng thái-hành động (s, a) có thể xuất hiện nhiều lần. Có hai biến thể chính để xử lý tình huống này:

First-Visit MC	Every-Visit MC
<i>First-Visit MC</i> chỉ sử dụng lần xuất hiện đầu tiên của (s, a) trong mỗi episode để	<i>Every-Visit MC</i> sử dụng tất cả các lần xuất hiện của (s, a) trong episode, tính

cập nhật $Q(s, a)$. Các lần xuất hiện tiếp theo bị bỏ qua.	return G_t tương ứng với từng lần và cập nhật độc lập.
---	--

Bảng 2: So sánh First Visit và Every Visit

Trong thực tiễn, First-Visit MC được ưa dùng hơn do bảo đảm tính không chệch và hội tụ lý thuyết. Thuật toán First-Visit MC Control hoàn chỉnh được mô tả như sau:

1. Khởi tạo tùy ý bảng $Q(s, a)$ và một chính sách ban đầu (ví dụ: ϵ -greedy).
2. Lặp lại cho mỗi episode:
 - a. Sử dụng chính sách hiện tại để Agent sinh ra một episode hoàn chỉnh: $(S_0, A_0, R_1, \dots, S_T)$.
 - b. Tính toán ngược G_t từ bước cuối cùng T về bước 0.
 - c. Duyệt qua chuỗi trạng thái từ đầu: Với mỗi cặp (S_t, SA_t) , kiểm tra xem nó đã từng xuất hiện ở các bước thời gian $0, 1, \dots, t-1$ trước đó trong episode hiện tại hay chưa.
 - d. Nếu chưa (đây là First-Visit), tiến hành cập nhật bảng Q cho cặp (S_t, A_t) dựa trên G_t vừa tính.
3. Cải thiện chính sách (Policy Improvement): Cập nhật lại chính sách ϵ -greedy dựa trên bảng Q mới.
4. Lặp lại quá trình cho đến khi chính sách hội tụ và tìm được đường đi tối ưu nhất đến đích.

1.5. Thuật toán Q-Learning

Q-Learning là một trong những thuật toán học tăng cường phi mô hình (model-free) phổ biến và đột phá nhất, được phát triển bởi Watkins vào năm 1989. Thuật toán này thuộc nhóm phương pháp Temporal-Difference (TD - Sai số thời gian), nổi bật với khả năng học được chính sách tối ưu (optimal policy) mà không cần Agent phải tuân thủ nghiêm ngặt theo chính sách đó trong lúc khám phá môi trường (Off-policy learning).

1.5.1. Cập nhật theo TD (Temporal-Difference)

Học theo Sai số thời gian (Temporal-Difference Learning) là sự kết hợp tinh tế giữa hai phương pháp: Monte Carlo và Quy hoạch động - Dynamic Programming.

Điểm khác biệt lớn nhất của TD so với Monte Carlo là thời điểm cập nhật kiến thức. Trong khi Monte Carlo yêu cầu Agent phải đi hết toàn bộ một tập huấn luyện mới

bắt đầu tính toán lại tổng phần thưởng, thì TD cho phép tác tử học ngay lập tức sau mỗi bước di chuyển đơn lẻ.

Ngay khi Agent thực hiện hành động A_t tại trạng thái S_t , nhận được phần thưởng R_{t+1} và quan sát được trạng thái mới S_{t+1} thuật toán TD sẽ lập tức sử dụng luồng thông tin $(S_t, A_t, R_{t+1}, S_{t+1})$ này để cập nhật bảng Q-Table. Khả năng cập nhật theo từng bước này cực kỳ hữu ích trong các môi trường mê cung nhiều tầng phức tạp.

1.5.2. Bootstrap và công thức TD target

Bootstrapping ám chỉ việc thuật toán cập nhật một dự đoán (giá trị Q hiện tại) dựa trên một dự đoán khác (giá trị Q của trạng thái tiếp theo) thay vì chờ đợi kết quả cuối cùng chính xác tuyệt đối.

Trong Q-Learning, mục tiêu mà thuật toán muốn hướng tới tại mỗi bước được gọi là TD Target (Mục tiêu TD). Nó bao gồm phần thưởng thực tế vừa nhận được cộng với giá trị kỳ vọng tốt nhất của trạng thái tiếp theo:

$$\text{TD Target} = R_{t+1} + \gamma \times \max_a Q(S_{t+1}, a)$$

Sự chênh lệch giữa mục tiêu này và giá trị Q hiện tại tạo ra Sai số TD (TD Error):

$$\text{TD Error} = \text{TD Target} - Q(S_t, A_t)$$

Từ đó, công thức cập nhật hoàn chỉnh của Q-Learning được phát biểu như sau:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \times [R_{t+1} + \gamma \times \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Trong đó:

- α (Alpha): Tốc độ học (Learning rate), xác định mức độ điều chỉnh giá trị cũ theo sai số mới.

Có thể phân tích trực quan theo công thức bên dưới ảnh:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

The diagram shows the Q-Learning formula with color-coded components and their meanings:

- $Q(S_t, A_t)$ (green underline): New Q-value estimation
- $Q(S_t, A_t)$ (blue underline): Former Q-value estimation
- α (red underline): Learning Rate
- R_{t+1} (orange underline): Immediate Reward
- $\gamma \max_a Q(S_{t+1}, a)$ (purple underline): Discounted Estimate optimal Q-value of next state
- $Q(S_t, A_t)$ (blue underline): Former Q-value estimation

The difference between the TD Target and the former Q-value estimation is labeled as TD Error (yellow underline).

Hình 2: Trực quan hóa công thức Q-Learning

1.5.3. Ưu/nhược điểm so với Monte Carlo

Ưu điểm:

- Học theo từng bước: Không cần chờ kết thúc episode.
- Xử lý được môi trường liên tục: Áp dụng tốt cả trong các bài toán không có trạng thái kết thúc rõ ràng.
- Phương sai thấp: Do chỉ phụ thuộc vào một bước phần thưởng ngẫu nhiên R_{t+1} , quá trình hội tụ của TD thường ổn định và trơn tru hơn.

Nhược điểm:

- Chênh ban đầu: Việc sử dụng Bootstrapping khiến các cập nhật ban đầu phụ thuộc vào các giá trị dự đoán sai lệch (thường khởi tạo bằng 0), có thể dẫn đến việc học các chiến lược cục bộ kém hiệu quả trong thời gian đầu.
- Nhạy cảm với tham số: Đòi hỏi phải tinh chỉnh kỹ lưỡng tốc độ học α để tránh việc bảng Q dao động không hội tụ.

1.6. Double Deep Q-Network (Double DQN)

Trong các môi trường có không gian trạng thái khổng lồ như mê cung nhiều tầng phức tạp, thuật toán Q-Learning truyền thống bộc lộ giới hạn nghiêm trọng do phải duy trì một bảng Q-Table khổng lồ, dẫn đến việc cạn kiệt bộ nhớ và thời gian hội tụ kéo dài. Double DQN ra đời như một bước đột phá bằng cách kết hợp sức mạnh biểu diễn đặc trưng của mạng thần kinh học sâu (Deep Learning) vào học tăng cường.

1.6.1. Xấp xỉ Q-function bằng Neural Network

Thay vì lưu trữ giá trị Q của từng cặp (trạng thái, hành động) trong một ma trận hai chiều, Double DQN sử dụng một mạng thần kinh nhân tạo (Neural Network) đóng vai trò là một bộ xấp xỉ hàm (Function Approximator), ký hiệu là $Q(s, a; \theta)$, với $Q(s, a; \theta)$ là tập hợp các trọng số (weights) của mạng.

Đầu vào (Input): Là một vector hoặc tensor đại diện cho trạng thái s của môi trường (ví dụ: tọa độ Agent, cấu trúc mê cung, vị trí quái vật, thông tin tầng).

Đầu ra (Output): Khác với xấp xỉ hàm thông thường, cấu trúc mạng của Double DQN nhận đầu vào là trạng thái s và xuất ra một vector chứa giá trị Q ước lượng cho toàn bộ các hành động a hợp lệ cùng một lúc.

1.6.2. Chu trình huấn luyện Double DQN

Quá trình huấn luyện Double DQN diễn ra qua một chu trình gồm ba pha chính:

1. Thu thập dữ liệu (Collect): Tác tử sử dụng chính sách hiện tại để tương tác với môi trường, thực hiện hành động a , nhận phần thưởng r và trạng thái mới s' .
2. Phát lại trải nghiệm (Replay): Thay vì học ngay, Agent đóng gói bước chuyển (transition) này thành một bộ giá trị (s, a, r, s') và lưu vào một bộ nhớ đệm. Sau đó, thuật toán sẽ bốc ngẫu nhiên một lô nhỏ (mini-batch) các trải nghiệm từ bộ nhớ này ra để chuẩn bị huấn luyện.
3. Cập nhật (Update): Tính toán sai số giữa giá trị Q dự đoán và giá trị mục tiêu (TD Target) dựa trên mini-batch vừa lấy ra, sau đó lan truyền ngược (backpropagation) để cập nhật trọng số θ của mạng.

1.6.3. Experience Replay Buffer

Bộ đệm phát lại trải nghiệm là một cấu trúc dữ liệu dạng hàng đợi vòng có kích thước cố định, dùng để lưu trữ N bước chuyển gần nhất của agent.

Việc sử dụng Replay Buffer giải quyết hai vấn đề lớn khi huấn luyện:

- Phá vỡ sự tương quan thời gian (Temporal Correlation): Trong một màn chơi, các trạng thái liên tiếp nhau thường có độ tương đồng rất cao. Nếu huấn luyện mạng bằng các mẫu dữ liệu theo thứ tự thời gian này, mạng sẽ dễ bị rơi vào trạng thái học lệch. Việc lấy mẫu ngẫu nhiên từ bộ đệm giúp dữ liệu huấn luyện mang tính độc lập và phân phối đồng đều hơn.
- Tái sử dụng dữ liệu (Sample Efficiency): Mỗi trải nghiệm đắt giá có thể được bốc ra huấn luyện nhiều lần thay vì dùng một lần rồi bỏ, giúp tăng tốc độ hội tụ.

1.6.4 Target Network (Soft update)

Trong Q-Learning chuẩn, giá trị mục tiêu được tính bằng công thức:

$$Y = R + \gamma \times \max Q(s', a'; \theta)$$

Vấn đề là trọng số θ liên tục thay đổi sau mỗi bước cập nhật, gây ra sự dao động và mất ổn định nghiêm trọng.

Double DQN giải quyết bằng cách tách thành hai mạng nơ-ron:

- Mạng chính (Main Network - θ): Mạng liên tục được cập nhật trọng số ở mỗi bước huấn luyện và dùng để chọn hành động.
- Mạng mục tiêu (Target Network - θ'): Mạng dùng để tính giá trị mục tiêu Y .

Cập nhật mềm (Soft Update): Thay vì sao chép toàn bộ trọng số từ mạng chính sang mạng mục tiêu sau mỗi N bước (Hard Update), kỹ thuật Soft Update sẽ trộn từ từ trọng số ở mỗi bước huấn luyện theo tham số τ (Tau), với τ rất nhỏ:

$$\theta' \leftarrow \tau \times \theta + (1 - \tau) \times \theta'$$

Bên cạnh đó, cơ chế tách mạng này còn khắc phục overestimation bias của DQN gốc. Trong DQN truyền thống, cùng một mạng vừa chọn hành động vừa đánh giá giá trị, dẫn đến xu hướng thổi phồng giá trị Q. Với Double DQN, θ chọn hành động còn θ' đánh giá giá trị của hành động đó. Vì hai mạng có trọng số khác nhau, sai số không còn cộng hưởng, từ đó giảm đáng kể overestimation bias.

1.6.5 Hàm mất mát (Huber / Smooth L1) và cập nhật gradient

Để đánh giá mức độ sai lệch của mạng, ta cần một hàm mất mát (Loss Function). Nếu sử dụng hàm sai số toàn phương trung bình (MSE - Mean Squared Error), khi Agent gặp những phần thưởng bất ngờ lớn (ví dụ tìm được kho báu), sai số TD Error sẽ rất lớn, kéo theo gradient bùng nổ (exploding gradients) làm phá hỏng trọng số mạng.

Hàm mất mát Huber (hay Smooth L1 Loss) được sử dụng để khắc phục:

- Khi sai số $|x| < 1$: Hàm hoạt động như MSE (bậc 2), $x^2 / 2$.
- Khi sai số $|x| \geq 1$: Hàm hoạt động như MAE (bậc 1), $|x| - 0.5$, làm giảm độ dốc, tránh việc gradient bị cập nhật quá mức bởi các giá trị dị thường (outliers).

Trọng số mạng θ sau đó được tối ưu hóa thông qua các thuật toán Gradient Descent để cực tiểu hóa hàm Huber Loss.

1.6.6 Action Masking

Action Masking là một kỹ thuật can thiệp vào quá trình chọn hành động của mạng thần kinh. Trước khi tác tử chọn hành động (thông qua argmax), ta áp dụng một mặt nạ lên vector Q-value đầu ra. Các hành động không hợp lệ sẽ bị ép giá trị Q xuống mức âm vô cùng. Khi đó hàm argmax sẽ tự động bỏ qua các hành động này và chỉ chọn hành động có giá trị Q cao nhất trong số các hành động hợp lệ.

CHƯƠNG 2: MÔ TẢ HỆ THỐNG

2.1. Thiết kế bài toán

2.1.1. Mô tả môi trường

Môi trường được mô hình hóa dưới dạng mê cung đa tầng (multi-floor maze), gồm 3 tầng liên tiếp. Mỗi tầng là một lưới hai chiều kích thước 7×7 ô, được sinh ngẫu nhiên tại thời điểm khởi tạo, reset. Tác tử bắt đầu từ tầng 0 và phải lần lượt đi qua từng tầng để đạt đến ô đích (GOAL) tại tầng 2.

Mỗi tầng được sinh bằng cách chọn ngẫu nhiên một trong sáu mẫu tường cấu hình sẵn - bao gồm các bức tường dọc và ngang ở các vị trí khác nhau trong lưới. Sau đó đặt một ô cửa ngẫu nhiên trên tường đó, phân chia tầng thành hai vùng riêng biệt. Quá trình sinh bản đồ thực hiện tối đa 200 lần thử; tính hợp lệ được xác minh bằng thuật toán BFS (Breadth-First Search) kiểm tra sự tồn tại của đường đi liên thông giữa các mốc bắt buộc theo chuỗi: ENTRY $\rightarrow$ KEY $\rightarrow$ DOOR $\rightarrow$ STAIRS_UP (hoặc GOAL tại tầng cuối). Toàn bộ bản đồ được sinh mỗi episode thông qua tham số `randomize_map`, nhằm đảm bảo tác tử không thể ghi nhớ cấu trúc cố định và buộc phải học một chính sách tổng quát.



Hình 3: 3 dạng tường dọc cấu hình sẵn



Hình 4: 3 dạng tường ngang cấu hình sẵn

Mỗi ô trong lưới mang một trong 13 giá trị nguyên, biểu diễn các thực thể khác nhau trong môi trường.

Ký hiệu	Giá trị	Mô tả
EMPTY	0	Ô trống, tác tử và quái đi qua được
WALL	1	Tường cứng, không thể vượt qua
KEY	2	Chìa khóa - điều kiện cần để mở cửa
BOMB	3	Bom - trừ máu khi bước vào
HOLE	4	Hố - trừ máu, teleport về ENTRY tầng
BLOOD	5	Máu hồi phục (spawn động)
DOOR	6	Cửa – tự mở được khi có KEY
STAIRS_UP	7	Cầu thang lên tầng tiếp theo
ENTRY	8	Điểm xuất phát của tầng (respawn point)
GOAL	9	Đích đến cuối cùng (tầng F-1)
MONSTER	10	Quái vật (quản lý động, không tĩnh trên map)
SHIELD	11	Khiên bảo vệ
SOFT_WALL	12	Tường mềm - có thể phá bằng búa

Bảng 3: Mô tả các thực thể trong môi trường

Ngoài tường cứng (WALL) không thể vượt qua, còn có thiết kế tường mềm (SOFT_WALL) - loại tường có thể bị phá hủy bởi búa của tác tử, biến thành ô trống và mở ra lối đi mới. Tường mềm được đặt vào các ô trống hợp lệ sau khi hoàn tất việc bố trí các vật phẩm bắt buộc, và không được đặt đè lên ô ENTRY để tránh chặn điểm xuất phát của tác tử.

Ngoài bản đồ tĩnh, môi trường duy trì một lớp thực thể động riêng biệt để quản lý quái vật, do vị trí của chúng thay đổi liên tục theo từng bước thời gian. Mỗi tầng có đúng một quái vật với vị trí spawn cố định. Hành vi của quái vật phụ thuộc vào khoảng cách Manhattan đến tác tử theo công thức với m là quái vật, a là tác tử $d(t) = (y_m - y_a) + (x_m - x_a) \leq V$: nếu khoảng cách này không vượt quá ngưỡng tầm nhìn bằng $V = 3$, quái vật chuyển sang chế độ truy đuổi - ưu tiên di chuyển theo hướng giảm khoảng cách đến tác tử, nhưng có xác suất 35% hụt bước mỗi lần; nếu khoảng cách vượt ngưỡng, quái vật di chuyển ngẫu nhiên với xác suất đứng yên 20%. Quái vật bị chặn bởi tường cứng, tường mềm, cửa, hố và bom, không thể di chuyển qua các ô này.

Tác tử có số máu (HP) tối đa bằng 3. Khi va chạm với quái vật hoặc rơi xuống hố mà không có khiên, tác tử mất 1 HP và bị dịch chuyển về ô ENTRY của tầng hiện tại, đồng thời quái vật cũng được hồi phục về vị trí spawn ban đầu; đối với trúng bom, tác tử chỉ mất 1 HP. Nếu HP giảm về 0, episode kết thúc ngay lập tức với kết quả thua. Khiên (SHIELD) là vật phẩm phòng thủ một lần: khi tác tử sở hữu khiên và chịu bất kỳ sát thương nào - từ quái vật, bom hoặc hố - khiên sẽ bị tiêu hao thay cho HP, đồng thời gây choáng (stun) quái vật trong 2 bước nếu nguồn sát thương là va chạm quái. Khiên bị xóa khi tác tử lên tầng mới.

Tác tử còn được trang bị búa (hammer), cho phép tấn công vào ô liền kề theo bốn hướng. Búa có thể gây choáng quái vật trong 2 bước, phá hủy tường mềm thành ô trống, hoặc xóa bom trước khi tác tử bước vào. Sau mỗi lần sử dụng, búa cần 3 bước thời gian để hồi chiêu - khoảng thời gian này dài hơn thời gian choáng một cách có chủ ý, nhằm ngăn chặn chiến lược lạm dụng búa liên tục để vô hiệu hóa quái vật.

Bên cạnh các vật phẩm được đặt cố định lúc sinh bản đồ, môi trường còn tích hợp cơ chế spawn động: sau mỗi 5 đến 7 bước thời gian (ngẫu nhiên), hệ thống tự động sinh thêm bom hoặc vật phẩm hồi phục máu vào một ô trống hợp lệ trên tầng hiện tại. Ô được chọn phải thỏa mãn đồng thời các điều kiện: không có tác tử hoặc quái vật đứng trên đó, cách tác tử tối thiểu 3 ô theo khoảng cách Manhattan, và cách các vật phẩm cùng loại hiện có tối thiểu 3 ô. Số lượng bom và máu tồn tại đồng thời trên mỗi tầng được giới hạn lần lượt ở một mức do chúng ta thiết lập, ví dụ số lượng máu tối đa là 5, bom là 2 thì trên tầng đó chỉ có tối đa là như vậy. Cơ chế này tạo ra sự bất định liên tục trong suốt một episode, buộc tác tử phải phản ứng với trạng thái môi trường thay đổi thay vì chỉ đơn thuần thực thi một lộ trình cố định.

2.1.2. Cấu hình môi trường

2.1.2.1. Trạng thái đầy đủ

Trạng thái đầy đủ của môi trường (full state) bao gồm toàn bộ thông tin nội tại của hệ thống tại một thời điểm bất kỳ, cụ thể gồm: ma trận bản đồ ba chiều của tất cả các tầng (`board[floor][row][col]`), vị trí hiện tại của tác nhân được biểu diễn bởi bộ ba (tầng, hàng, cột), điểm máu hiện tại (`hp`), trạng thái sở hữu chìa khóa ($\{0, 1\}$), trạng thái sở hữu khiên ($\{0, 1\}$), điểm tích lũy (`score`), bộ đếm hồi chiêu của búa (`hammerTimer`),

và danh sách trạng thái của từng quái vật trên tất cả các tầng (vị trí, số bước choáng còn lại, cờ đang truy đuổi). Full state chủ yếu phục vụ mục đích hiển thị giao diện UI.

2.1.2.2. Trạng thái quan sát được

Do không gian trạng thái đầy đủ có kích thước rất lớn và không phù hợp để đưa trực tiếp vào nên chúng ta đưa cho tác tử trạng thái quan sát được (observable state). Đây là một tuple 19 chiều, nhằm cô đọng những thông tin thiết yếu nhất phục vụ việc ra quyết định, đồng thời đủ gọn để học hiệu quả.

Toàn bộ vector quan sát được biểu diễn dưới dạng tuple gồm 19 phần tử:

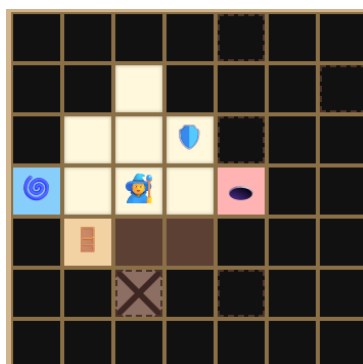
$s = (\text{floor}, \text{hp_state}, \text{key}, \text{shield},$
 $\text{cell_up}, \text{cell_down}, \text{cell_left}, \text{cell_right},$
 $\text{cell_up2}, \text{cell_down2}, \text{cell_left2}, \text{cell_right2},$
 $\text{cell_ul}, \text{cell_ur}, \text{cell_dl}, \text{cell_dr},$
 $\text{monster_cell_index}, \text{monster_stun_flag}, \text{hammer_ready}).$

19 giá trị này được mô tả trực tiếp bên dưới đây:

Trạng thái này được phân tách thành ba nhóm đặc trưng có ngữ nghĩa khác biệt. Đối với Double DQN thì chia thành 3 nhóm vector, đối với Monte Carlo Control và Q-Learning thì 3 nhóm này gộp lại thành 1 tuple:

Nhóm 1 - Đặc trưng cục bộ (Local features, 12 chiều, chỉ số 4–15): Nhóm này mã hóa loại ô tại 12 vị trí xung quanh tác nhân theo hình kim cương mở rộng, bao gồm:

- 4 ô kề trực tiếp: trên, dưới, trái, phải có chỉ số 4, 5, 6, 7.
- 4 ô kề cách 2 bước theo trục ngang và dọc: trên 2, dưới 2, trái 2, phải 2 có chỉ số 8, 9, 10, 11.
- 4 ô chéo góc: trên-trái, trên-phải, dưới-trái, dưới-phải có chỉ số 12, 13, 14, 15.



Hình 5: Minh họa 12 ô quan sát được

Mỗi giá trị trong nhóm này là một số nguyên trong phạm vi $[0, 12]$ tương ứng với một trong 13 loại ô được định nghĩa trong môi trường. Nhóm đặc trưng cục bộ cung cấp nhận thức không gian lân cận, giúp tác nhân hiểu được địa hình, nguy hiểm và phần thưởng tiềm năng trong tầm nhìn gần.

Nhóm 2 - Đặc trưng tương tác (Interaction features, 4 chiều, chỉ số 1, 2, 3, 18):

Nhóm này mã hóa trạng thái nội tại của tác nhân liên quan đến khả năng tương tác với môi trường, gồm:

- `hp_state` (có chỉ số là 1): trạng thái máu được rời rạc hóa - nhận giá trị 0 nếu $HP \leq 1$ (nguy hiểm) và 1 nếu $HP \geq 2$ (an toàn). Việc rời rạc hóa thay vì dùng giá trị HP thực giúp thu gọn không gian trạng thái, vì điều tác nhân cần phân biệt chủ yếu là "sắp chết" hay "chưa nguy hiểm", không cần biết chính xác còn bao nhiêu máu.
- `key` (có chỉ số là 2): cờ nhị phân biểu thị tác nhân đang giữ chìa khóa hay không (0/1), quyết định khả năng mở cửa DOOR;
- `shield` (có chỉ số là 3): cờ nhị phân biểu thị tác nhân đang có khiên hay không (0/1), quyết định khả năng chặn sát thương;
- `hammer_ready` (có chỉ số là 18): cờ nhị phân biểu thị búa có sẵn sàng sử dụng hay không (1 nếu `hammerTimer` bằng 0, 0 nếu còn trong thời gian hồi chiêu).

Nhóm 3 - Đặc trưng ngữ cảnh (Context features, 3 chiều, chỉ số 0, 16, 17): Nhóm này cung cấp thông tin toàn cục và thông tin về mối đe dọa động, gồm:

- `floor` (có chỉ số là 0): chỉ số tầng hiện tại của tác nhân (0, 1, ..., tổng tầng - 1), cho phép tác nhân nhận thức được mình đang ở giai đoạn nào của nhiệm vụ.
- `monster_cell_index` (có chỉ số là 16): vị trí quái vật trong vùng 12 ô xung quanh - nhận giá trị 0 nếu không có quái trong tầm, 1-12 nếu quái nằm ở ô thứ tương ứng trong 12 ô theo thứ tự đã định nghĩa, và 13 nếu quái đang trùng ô với tác tử.
- `monster_stun_flag` (có chỉ số là 17): cờ nhị phân biểu thị quái vật trên tầng hiện tại có đang trong trạng thái choáng hay không (0/1). Thông tin này có ý nghĩa chiến thuật quan trọng vì khi quái đang choáng, tác tử có thể đi qua mà không bị sát thương.

2.1.3. Không gian hành động (Action Space)

Tập hành động đầy đủ A gồm 9 hành động rời rạc:

$$A = \{0,1,2,3,4,5,6,7,8\}$$

0	Di chuyển lên (Up)	5	Dùng búa hướng phải
1	Di chuyển phải (Right)	6	Dùng búa hướng xuống
2	Di chuyển xuống (Down)	7	Dùng búa hướng trái
3	Di chuyển trái (Left)	8	Đứng yên (Wait)
4	Dùng búa hướng lên		

Bảng 4: Mô tả các hành động

2.1.4. Hệ thống phần thưởng (Reward Function)

Hàm phần thưởng được thiết kế theo nguyên tắc reward shaping có chủ đích, kết hợp giữa phần thưởng thừa thớt và dày đặc, nhằm dẫn hướng tác tử học tuần tự theo chuỗi mục tiêu của bài toán.

Sự kiện	Giá trị	Ghi chú
Mỗi bước đi hợp lệ	-0.3	Khuyến khích giải nhanh
Đứng yên (wait)	-0.5	Phạt nặng hơn bước đi thường
Hành động không hợp lệ	-3	Ngăn chặn tương
Nhặt chìa khóa (KEY)	35	Cho tác tử biết là nên làm
Mở cửa (DOOR)	45	Cho tác tử biết là nên làm
Lên tầng (STAIRS_UP)	60	Cho tác tử biết là nên làm
Đạt đích (GOAL)	700	Kết thúc thắng
Trúng bom (BOMB)	-12	Trừ 1 máu + phạt điểm
Rơi hố (HOLE)	-28	Trừ 1 máu + teleport ENTRY
Bị quái đánh (MONSTER)	-22	Trừ 1 máu + respawn
Chết (hp = 0)	-520	Kết thúc thua
Hết bước (timeout)	-220	Kết thúc thua
Trì trệ (stagnation)	-180	Kết thúc thua sớm
Nhặt khiên (SHIELD)	10	Chỉ khi shield = 0
Chặn đòn bằng khiên	5	Tiêu thụ khiên, choáng quái
Nhặt máu (BLOOD)	1	Chỉ khi hp < 3
Búa trúng quái	0.5	Chỉ khi quái chưa choáng
Búa phá tường mềm	1	Xóa SOFT_WALL

Búa phá bom	0.5	Xóa BOMB
Búa trượt (nothing)	-3	Ngăn spam búa vô nghĩa
Hệ số mới lạ (Novelty bonus)	0.3	Khám phá ô mới
Hệ số phạt lặp lại (Repeat visit penalty)	0.3	Một thành phần trừ của hàm đánh giá
Ngưỡng lặp lại (Repeat visit threshold)	3	Số lần tối đa một vị trí có thể được ghé thăm

Bảng 5: Mô tả chi tiết thưởng/phạt

Reward hacking là hiện tượng tác tử tìm ra các chiến lược khai thác bất thường trong hàm phần thưởng để tối đa hóa điểm số mà không thực sự hoàn thành mục tiêu thiết kế. Với cấu hình phần thưởng hiện tại, có thể nhận diện một số rủi ro tiềm ẩn và các biện pháp đối phó tương ứng như sau:

Khai thác khiên để farm điểm: Tác tử có thể học cách cố ý để bị quái tấn công khi đang có khiên nhằm lặp đi lặp lại việc thu thập shield_block = +5 mà không tiến triển nhiệm vụ. Biện pháp đối phó chính là cơ chế stagnation early-stop: nếu tác tử không đạt được bất kỳ mốc tiến trình nào (key, door, stairs, goal) trong 95 bước (38% tổng số bước tối đa) liên tiếp, episode bị kết thúc cưỡng bức với phạt stagnation = -180. Mức phạt này vượt xa tổng điểm có thể tích lũy từ farming khiên trong cùng khoảng thời gian đó, làm cho chiến lược khai thác này không có lợi về mặt kỳ vọng.

Lạm dụng búa để choáng quái lặp lại: Nếu phần thưởng đánh trúng quái không có giới hạn, tác tử có thể học cách liên tục dùng búa choáng cùng một con quái thay vì di chuyển về phía mục tiêu. Biện pháp đối phó là cơ chế hammer cooldown kết hợp thời gian choáng: búa cần hồi hammerCooldown = 3 bước sau mỗi lần sử dụng, trong khi quái chỉ bị choáng monsterStun = 2 bước. Quan trọng hơn, hammerCooldown > monsterStun được thiết kế có chủ đích - khi búa đã hết hồi (bước 3), quái đã hết choáng từ bước trước đó, nghĩa là không thể liên tục nhận thưởng hammer_monster mà không chịu rủi ro thực sự từ quái. Ngoài ra, kiểm tra trước khi cộng thưởng, đảm bảo chỉ thưởng khi choáng quái từ trạng thái không choáng, loại bỏ khả năng choáng liên tục khi búa vào quái đang đã choáng sẵn.

Tránh kết thúc episode để tích lũy điểm nhỏ: Với step = -0.3 và các phần thưởng dương nhỏ (máu, búa, khiên), tác tử về lý thuyết có thể cố kéo dài episode hơn là kết

thúc nhanh. Biện pháp đối phó là hai cơ chế kết thúc cưỡng bức: timeout = -220 khi vượt 250 bước và stagnation = -180 khi trì trệ, cả hai đều đủ lớn để làm cho việc kéo dài episode không có lợi so với việc hoàn thành nhiệm vụ nhận goal = +700.

Nhặt máu khi đang đầy máu: Phần thưởng chỉ được cộng khi hp < maxHP, tức là tác tử không nhận bất kỳ điểm nào nếu đứng trên ô máu trong khi đang đầy máu. Điều này ngăn chặn chiến lược giả mạo đơn giản là di chuyển qua lại trên ô máu.

2.1.5. Điều kiện kết thúc (Terminal Conditions)

Một episode kết thúc khi xảy ra một trong bốn trường hợp sau:

Thứ nhất là thắng (goal): tác tử đến được đích.

Thứ hai là chết (death): HP giảm về 0 do va chạm quái, trúng bom, hoặc rơi xuống hố khi không có khiên,...

Thứ ba là hết bước (timeout): tác tử đạt đến giới hạn 250 bước mà chưa đến được đích.

Thứ tư là trì trệ (stagnation): tác tử không đạt được bất kỳ mốc tiến độ nào. Ví dụ như không nhặt chìa khóa, mở cửa, lên tầng, hoặc đến đích - trong một khoảng thời gian liên tiếp vượt ngưỡng, được tính bằng 38% tổng số bước tối đa (tương đương khoảng 95 bước). Cơ chế stagnation giúp rút ngắn các episode không hiệu quả trong quá trình huấn luyện, tránh lãng phí tài nguyên tính toán vào những lần chạy mà tác tử đã rõ ràng không còn tiến triển.

2.2. Cài đặt và triển khai

2.2.1. Sơ đồ khối hệ thống và kiến trúc mã nguồn

Gồm ba lớp chức năng chính: môi trường mô phỏng, khối huấn luyện thuật toán, và khối đánh giá/lưu trữ kết quả. Kiến trúc này giúp phân biệt phần logic game với phần học tăng cường, thuận tiện cho việc thay thế thuật toán và so sánh thực nghiệm.

Luồng xử lý chính có thể mô tả như sau:

1. Khởi tạo môi trường: Sinh bản đồ đa tầng và vị trí xuất phát. Thiết lập reward setting và tham số động học của môi trường.
2. Khởi tạo agent theo thuật toán: Q-learning (tabular), Monte Carlo Control (first-visit), Double DQN (network + replay + target).
3. Vòng lặp huấn luyện episode: Môi trường reset theo chế độ map (fixed/random). Agent chọn hành động theo policy hiện tại. Môi trường trả

về state tiếp theo, reward, done. Agent cập nhật tham số (bảng Q hoặc mạng nơ-ron). Ghi log theo episode.

4. Lưu checkpoint và mô hình tốt nhất: Lưu định kỳ để theo dõi quá trình học. Lưu lại cho việc theo dõi tiêu chí thực nghiệm.
5. Đánh giá (evaluate): Chạy policy greedy đối với mô hình đã huấn luyện. Tổng hợp chỉ số: reward trung bình, tỉ lệ goal, số bước, end_reason.

2.2.2. Monte Carlo (cài đặt tabular)

Thuật toán Monte Carlo trong đồ án được cài đặt theo hướng First-Visit Monte Carlo Control, tức là cập nhật giá trị hành động dựa trên return của toàn bộ episode và chỉ cập nhật cho lần xuất hiện đầu tiên của mỗi cặp (s, a) trong episode đó.

Thay vì lưu trực tiếp giá trị trung bình $Q(s, a)$, MCTable lưu hai thành phần: Tổng return tích lũy cho mỗi (s, a), số lần quan sát $N(s, a)$.

Nếu (s, a) chưa từng xuất hiện, trả về giá trị mặc định defaultQ.

```
def __init__(self, default):
    dict.__init__(self)
    self.nCounts = {}
    self.default = default

def __getitem__(self, k):
    if super().__contains__(k):
        return super().__getitem__(k) / self.nCounts[k]
    return self.default

def __setitem__(self, k, v):
    if super().__contains__(k):
        oldV = super().__getitem__(k)
    else:
        oldV = 0
    self.nCounts[k] = 0
    super().__setitem__(k, oldV + v)
    self.nCounts[k] += 1
```

Hình 6: Code MCTable

Monte Carlo control trong đồ án dùng ϵ -greedy tương tự các thuật toán khác:

```
if len(indexes) == len(qs):
    p = [1.0 / len(indexes)] * len(qs)
else:
    p = [
        (1.0 - self.epsilon) / len(indexes) if i in indexes
        else self.epsilon / (len(qs) - len(indexes))
        for i in range(len(qs))
    ]
return int(self.randGen.choice(actions, p=p))
```

Hình 7: Code ϵ -greedy trong Monte Carlo

Sau mỗi episode, ϵ giảm theo decay để chuyển dần từ khám phá sang khai thác, đồng thời giữ sàn $\epsilon_{\min}$.

```
def decay_epsilon(self):
    # Sau mỗi episode: giảm epsilon nhưng không thấp hơn epsilon_min
    self.epsilon = max(self.epsilon_min, self.epsilon * self.decay_rate)
```

Hình 8: Code ϵ decay trong Monte Carlo

Sau khi episode kết thúc, thuật toán tính return lùi từ cuối episode:

$$G_t = r_{t+1} + \gamma G_{t+1}$$

Sau đó duyệt trajectory từ đầu đến cuối và chỉ cập nhật (s_t, a_t) nếu đây là lần đầu tiên cặp này xuất hiện trong episode hiện tại. Cập nhật thực hiện bằng cách cộng (G_t) vào tổng và tăng count. Ý tưởng first-visit giúp giảm lệch do một cặp (s, a) lặp nhiều lần trong cùng episode.

```
def improve(self, trajectory):
    # Cập nhật Q bằng first-visit Monte Carlo
    # trajectory[t] = (s_t, a_t, r_{t+1}):
    # s_t: state trước khi gọi step(a_t)
    # r_{t+1}: reward env trả về sau step đó

    # Return tại bước t: G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 ... đến hết episode
    # First-visit: với mỗi (s,a), chỉ dùng G của lần đầu tiên (s,a) xuất hiện trong episode

    n = len(trajectory)
    if n == 0:
        return

    # Bước 1: tính G_t cho mọi t bằng công thức lùi G_t = r_{t+1} + \gamma * G_{t+1}
    G = 0.0
    Gs = [0.0] * n
    gamma = self.delta
    for t in range(n - 1, -1, -1):
        G = trajectory[t][2] + gamma * G
        Gs[t] = G

    # Bước 2: first-visit - chỉ table[s,a] += G_t khi (s,a) chưa gặp trong episode
    seen = set()
    for t in range(n):
        s, a, _ = trajectory[t]
        key = (s, a)
        if key in seen:
            continue
        seen.add(key)
        self.table[key] = Gs[t]
```

Hình 9: Code hàm improve của Monte Carlo

Vòng train MC gồm:

1. Reset môi trường theo fixed map hoặc random map.
2. Chạy trọn vẹn episode và lưu trajectory.
3. Tính (G_t) cho từng thời điểm.
4. Cập nhật first-visit vào MCTable.
5. Decay epsilon.
6. Ghi log reward, success_rate, số bước, end_reason, tăng kết thúc.

2.2.3. Q-learning (cài đặt tabular)

Trong đồ án, Q-learning được cài đặt theo hướng tabular off-policy, sử dụng bảng giá trị $Q(s,a)$ để học trực tiếp chính sách tối ưu từ tương tác với môi trường. Giá trị mặc định của cặp chưa từng gặp được gán qua defaultQ.

```
# QTable của QLearning được lưu ở dạng dict gồm key-value với key = (state, action) || value = Q(s,a)
# State bao gồm x số, action là 1 số đại diện cho 1 hành động -> Trả về value của hành động đó là Q
class QTable_QLearning(dict):
    def __init__(self, default):
        dict.__init__(self)
        self.default = default

    def __getitem__(self, k):
        return super().get(k, self.default)

    def __setitem__(self, k, v):
        super().__setitem__(k, v)

# Ở trạng thái state s, thì các actions mỗi action sẽ có một Q
# Chọn ra và trả về giá trị Q = value lớn nhất -> Dùng cập nhật Q value khi train theo policy
def max(self, s, actions):
    if not actions:
        return self.default
    return max(self[s, a] for a in actions)
```

Hình 10: Code Q-table

Q-learning sử dụng ϵ - greedy trên tập hành động hợp lệ. Nếu nhiều hành động cùng Q-max, agent chọn theo phân phối đều giữa các hành động tốt nhất.

Sau mỗi episode, ϵ được giảm theo decay và chặn dưới bởi $\epsilon_{\min}$ để duy trì mức khám phá tối thiểu.

```
def decay_epsilon(self):
    # Giảm epsilon nhưng không được thấp hơn mức tối thiểu
    self.epsilon = max(self.epsilon_min, self.epsilon * self.decay_rate)
```

Hình 11: Code ϵ decay trong Q-Learning

Tại mỗi bước, sau khi thực hiện a_t ở trạng thái s_t , nhận reward r_{t+1} và trạng thái kế tiếp s_{t+1} , Q-learning cập nhật (với α là learning rate, γ là discount factor):

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')]$$

Nếu episode kết thúc tại bước đó, thành phần tương lai bị loại bỏ, còn lại:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha \cdot r_{t+1}$$

```

# Cập nhật Q(st,at) theo công thức Q-learning
def improve(self, st, at, s_next, actions_next, r, done):
    if done:
        # Nếu game kết thúc thì giá trị tương lai/ reward tương lai = 0 nên ko
        tdTarget = r
    else:
        # env hiện đang ở s_t+1 nên các actions possible at+1 sẽ theo s_t+1
        # Tìm Max Q của trạng thái kế tiếp/ giá trị tương lai
        tdTarget = r + self.delta * self.table.max(s_next, actions_next)

    old_q = self.table[st, at]
    self.table[st, at] = (1 - self.alpha) * old_q + self.alpha * tdTarget

```

Hình 12: Code hàm improve trong Q-Learning

Cơ chế này cho phép học online từng bước và lan truyền giá trị đích ngược về các trạng thái trước đó, giúp cho tác tử cập nhật giá trị Q sau mỗi step.

Mỗi episode huấn luyện gồm các bước:

1. Reset môi trường theo chế độ map (cố định hoặc ngẫu nhiên).
2. Lấy trạng thái hiện tại.
3. Chọn hành động bằng ϵ -greedy.
4. Gọi env.step(a) để nhận reward và trạng thái kế tiếp.
5. Cập nhật Q theo công thức TD.
6. Lặp cho đến khi episode kết thúc.
7. Giảm ϵ theo decay.

2.2.4. Kiến trúc mạng Double DQN

Trong đồ án, Double DQN được cài đặt nhằm xấp xỉ hàm giá trị hành động $Q(s,a)$ khi không gian trạng thái lớn và khó lưu trữ đầy đủ theo bảng. Double DQN học tham số mạng nơ-ron để tổng quát hóa giữa các trạng thái tương tự.

Trạng thái môi trường được mã hóa thành vector 19 chiều và tách thành ba nhóm đặc trưng. Cách tách này giúp mạng học theo cấu trúc ngữ nghĩa của trạng thái thay vì xem toàn bộ vector như một khối đồng nhất.

```

def split_state_vec(state_vec):
    # Split vector 19 chiều -> local(12), interaction(4), context(3)
    vec = np.asarray(state_vec, dtype=np.float32).reshape(-1)
    if vec.shape[0] != DQNStateProcessor.STATE_DIM:
        raise ValueError(f"split_state_vec: expected state dim {DQNStateProcessor.STATE_DIM}, got {vec.shape[0]}")
    local = vec[DQNStateProcessor.LOCAL_START_IDX:DQNStateProcessor.LOCAL_END_EXCLUSIVE_IDX]
    # One-hot hóa 12 ô local (mỗi ô 13 loại)
    local_idx = np.clip(np rint(local).astype(np.int64), 0, DQNStateProcessor.CELL_TYPE_COUNT - 1)
    local_onehot = np.eye(DQNStateProcessor.CELL_TYPE_COUNT, dtype=np.float32)[local_idx].reshape(-1)
    interaction = vec[DQNStateProcessor.INTERACTION_IDXS]
    context = vec[DQNStateProcessor.CONTEXT_IDXS]
    return local_onehot, interaction, context

```

Hình 13: Code mã hóa vector

Mạng Q sử dụng kiến trúc đa nhánh (multi-branch MLP):

1. Nhánh local: Linear(12 $\rightarrow$ hidden) + ReLU
2. Nhánh interaction: Linear(4 $\rightarrow$ hidden) + ReLU
3. Nhánh context: Linear(3 $\rightarrow$ hidden) + ReLU
4. Ghép ba nhánh, đưa qua lớp đầu ra Linear(3*hidden $\rightarrow$ 9)

Đầu ra của mạng là vector $Q(s, \cdot)$ gồm 9 giá trị, tương ứng 9 hành động rời rạc của môi trường. Kiến trúc đa nhánh cho phép mô hình học biểu diễn chuyên biệt cho từng loại đặc trưng trước khi tổng hợp quyết định hành động.

Do môi trường có tập hành động hợp lệ phụ thuộc trạng thái, Double DQN áp dụng action mask:

- Các action invalid bị che bằng giá trị rất nhỏ $\approx -\infty$ trước khi chọn argmax.
- Trong chế độ ϵ -greedy:
 - + explore: random trong tập hành động hợp lệ.
 - + exploit: chọn action hợp lệ có Q cao nhất.

Mỗi transition $(s, a, r, s', done)$ được lưu vào replay buffer dung lượng lớn. Khi học, mô hình lấy minibatch ngẫu nhiên từ buffer để: giảm tương quan chuỗi thời gian của dữ liệu, tái sử dụng trải nghiệm cũ và tăng ổn định gradient.

Mạng online và target network: Hệ thống dùng hai mạng:

- Online network: dùng để ước lượng $Q(s, a)$ và cập nhật gradient.
- Target network: dùng để tạo giá trị mục tiêu ổn định hơn.

Giá trị mục tiêu theo dạng:

$$Y = r + \gamma(1 - done)Q_{target}(s', \argmax_{a'} Q_{online}(s', a'))$$

Trong đồ án, target network được cập nhật theo soft update:

$$\theta_{tar_get} \leftarrow \tau \theta_{online} + (1 - \tau)\theta_{tar_get}$$

Soft update giúp target thay đổi mượt, giảm dao động học.

Sai số giữa $Q_{online}(s, a)$ và target y được tối ưu bằng:

- Huber loss (Smooth L1): bền vững hơn MSE khi có outlier.
- Adam optimizer: cập nhật tham số hiệu quả trong không gian phi tuyến.
- Gradient clipping: giới hạn norm gradient và tăng ổn định hội tụ.

```

# Q_pred(s,a)
q_pred = self.online_net(local_t, inter_t, ctx_t) # (batch, action_dim)
q_sa = q_pred.gather(1, actions_t.unsqueeze(1)).squeeze(1) # (batch,)

# Q_target(s',a') với mask action hợp lệ
with torch.no_grad():
    next_local_t, next_inter_t, next_ctx_t = DQNStateProcessor.split_state_batch_torch(next_states_t)

    q_next = self.target_net(next_local_t, next_inter_t, next_ctx_t) # (batch, action_dim)
    q_next_masked = q_next.masked_fill(~next_mask_t, -1e9)
    max_next_q = q_next_masked.max(dim=1).values # (batch,)

    target_q = rewards_t + self.gamma * (1.0 - dones_t) * max_next_q

loss = F.smooth_l1_loss(q_sa, target_q, beta=self.huber_delta)

self.optim.zero_grad(set_to_none=True)
loss.backward()
# Gradient clipping để ổn định (không bắt buộc nhưng thường hữu ích)
torch.nn.utils.clip_grad_norm_(self.online_net.parameters(), 1.0)
self.optim.step()

self.train_steps += 1
# Soft update duy nhất: target = tau * online + (1 - tau) * target
with torch.no_grad():
    for t_param, o_param in zip(self.target_net.parameters(), self.online_net.parameters()):
        t_param.data.copy_(self.tau * o_param.data + (1.0 - self.tau) * t_param.data)

```

Hình 13: DQN update step

2.2.5. Cấu hình tham số huấn luyện

Các thuật toán cùng huấn luyện trên một môi trường MazeEnv, do đó chia sẻ các thành phần cấu hình nền:

TOTAL_FLOORS: số tầng của mê cung là 3, maxSteps là 250, Seed là 42

- fixed_map / random_map:
 - fixed_map=True: cố định layout sau episode đầu.
 - fixed_map=False: random layout mỗi episode.

Tham số huấn luyện cho Q – Learning

<pre> # Map cố định: 50k ep agent_QLearning = QLearnir maze, lrate=0.08, discount=0.98, # Đã t epsilon=1.0, epsilon_min=0.05, decay_rate=0.99992, defaultQ=0, maxSteps=250, seed=42) </pre>	<pre> # Map random: 50k ep agent_QLearning = QLearnir maze, lrate=0.06, discount=0.98, epsilon=1.0, epsilon_min=0.1, # Rar decay_rate=0.9999385, defaultQ=0, maxSteps=300, seed=42) </pre>
--	---

Hình 14: Tham số huấn luyện Q-Learning

Tham số huấn luyện Monte Carlo

```

# Map cố định: 50k
agent_MC = MonteCarloCont
    maze,
    discount=0.98,
    epsilon=1.0, # explo
    epsilon_min=0.05,
    # decay_rate: decay =
    # epsilon chậm min qu
    decay_rate=0.99992,
    defaultQ=0.0,
    maxSteps=250,
    seed=42, # RNG epsil
)

# Map random: 50k
agent_MC = MonteCarloCont
    maze,
    discount=0.98,
    epsilon=1.0,
    epsilon_min=0.1, # r
    decay_rate=0.9999385,
    defaultQ=0.0,
    maxSteps=300,
    seed=42,
)

```

Hình 15: Tham số huấn luyện Monte Carlo

Tham số huấn luyện Double DQN

```

DQN_EP_AMOUNT = 20000
DQN_EP_MIN = 0.1

dqn_agent = DQNAgent(
    state_dim=DQNStateProcessor.STATE_DIM, # BẮT BUỘC khớp shape state hiện tại của env
    gamma=0.98, # Discount factor (ưu tiên reward tương lai)
    lr=3e-4, # Learning rate của optimizer
    hidden_dim=128, # Số neuron mỗi nhánh MLP
    buffer_capacity=200000, # Replay buffer lớn -> mẫu đa dạng hơn nhưng tốn RAM hơn
    batch_size=64, # Batch lớn ổn định hơn nhưng chậm/đòi GPU-RAM hơn
    tau=0.005, # Soft update: nhỏ -> target ổn định hơn, lớn -> bám online nhanh hơn

    # epsilon_start chỉ có hiệu lực khi train mới (không load checkpoint)
    # Nếu có load_path thì epsilon có thể bị ghi đè bởi checkpoint và/hoặc epsilon_override bên dưới
    epsilon_start=1.0,
    epsilon_min=DQN_EP_MIN,
    epsilon_decay=float(np.power(DQN_EP_MIN, 1.0 / int(DQN_EP_AMOUNT * 0.75))),
)

dqn_agent, logs = train_dqn_on_env(
    dqn_agent=dqn_agent,
    maze=maze,
    episodeAmount=DQN_EP_AMOUNT,
    fixed_map=True, # True: 1 layout map cố định (ổn định benchmark); False: random map mỗi ep
    seed=42, # Seed cho chuỗi env_seed (để tái lập kết quả)
    maxSteps=100, # Giới hạn bước/episode; timeout khi chậm ngưỡng
    printAfterEpisode=int(DQN_EP_AMOUNT * 0.04),
    saveDQNModel=True, # True để lưu checkpoint/best/final
    save_dir="/content/drive/MyDrive/DQN_Phase1_5_2_1_fixed", # Thư mục lưu model khi saveDQNModel=True
    plot_metrics=True, # Vẽ biểu đồ sau train
    window=50, # Cửa sổ moving average cho metric/plot
    # load_path != None: resume từ checkpoint, model/optimizer/epsilon có thể được nạp từ file.
    load_path=None, # Ví dụ: "DQN_Phase1/best_dqn.pt" để train tiếp
    load_map_location="cpu", # 'cpu' hoặc 'cuda'/'cuda:0'
    # epsilon_override ưu tiên cao nhất: set lại epsilon sau khi load (reheat exploration), nếu ko load thì ko set
    epsilon_override=None,
    learn_every=4,
)

```

Hình 16: Tham số huấn luyện Double DQN

2.3. Kết quả thực nghiệm và đánh giá

2.3.1. Thiết lập thực nghiệm và tiêu chí đánh giá

Để giảm sai lệch do ngẫu nhiên, quá trình thực nghiệm tuân theo các nguyên tắc: Dùng cùng môi trường MazeEnv với cùng bộ reward setting đã chốt. Thống nhất cấu hình bản đồ theo từng kịch bản. Thống nhất giới hạn bước mỗi episode để tránh lợi thế

thời gian không công bằng. Thiết lập seed 42 cho môi trường và RNG của agent để tăng khả năng tái lập.

Các chỉ số được dùng xuyên suốt:

1. Average Reward (avg_reward): Trung bình tổng reward theo cửa sổ episode.
2. Success Rate: Tỷ lệ episode kết thúc bằng goal.
3. Average Steps (avg_steps): Số bước trung bình/episode.
4. Phân bố end_reason: Tỷ lệ hoặc số lượng goal, death, timeout, stagnation.
5. Phân bố tầng kết thúc (end_floor distribution): xem agent đang kẹt ở tầng nào hay đã tiến sâu đến tầng cuối.

2.3.2. Biểu đồ hội tụ

Return theo episode (Reward Curve): Trục hoành: episode, trục tung: tổng reward mỗi episode. Kèm đường làm mượt (moving average) để giảm nhiễu.

Episode Length theo episode (Số bước/episode): Theo dõi độ dài hành trình agent. Giảm steps đi kèm tăng success thường là tín hiệu policy hiệu quả hơn.

Success rate theo cửa sổ episode: Phản ánh trực tiếp xác suất hoàn thành nhiệm vụ (end_reason = goal).

Phân bố end_reason và end_floor theo thời gian: Biểu thị tầng kết thúc và các kết thúc của một episode.

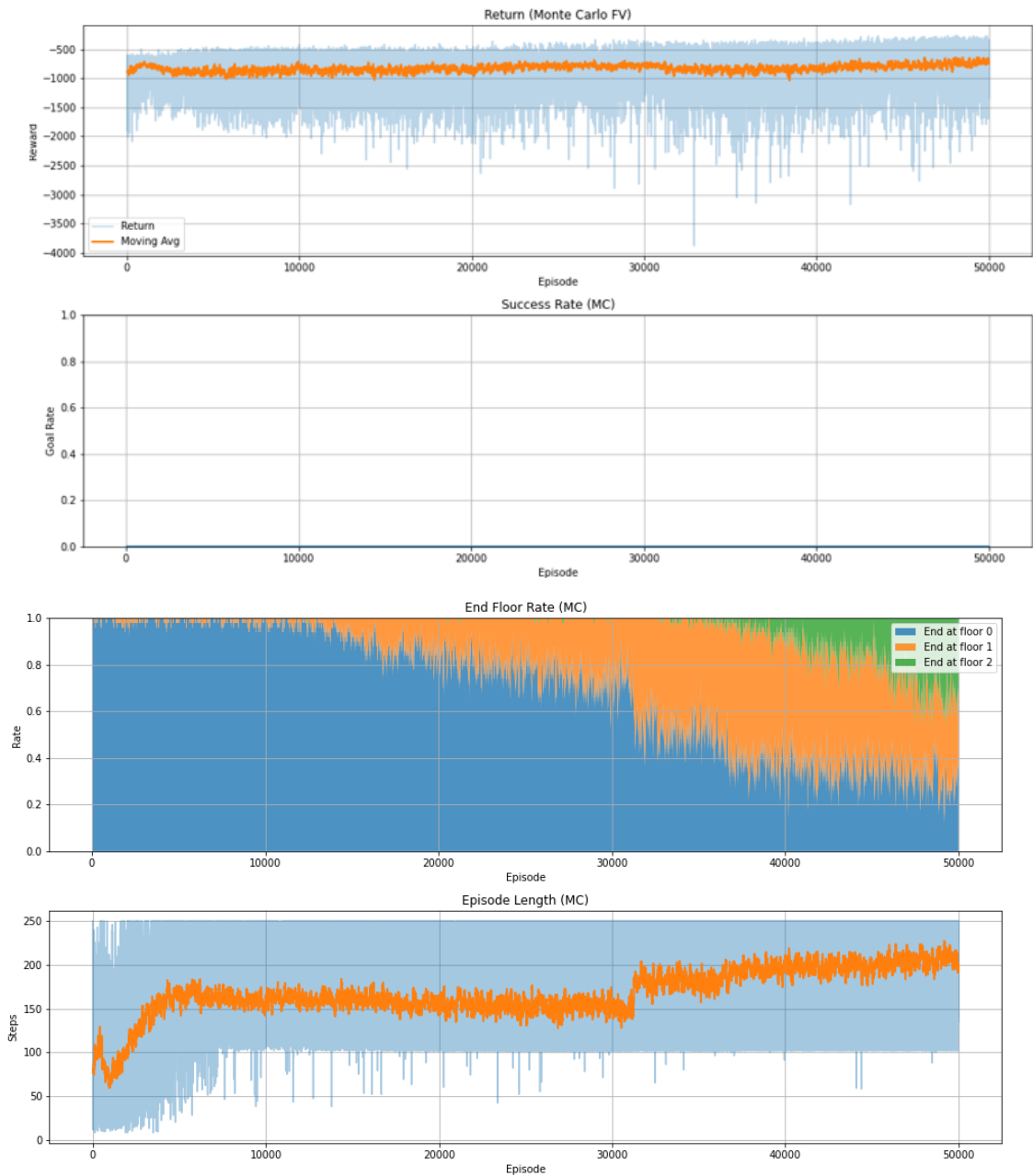
2.3.3. So sánh Q-learning, Monte Carlo

So sánh giữa Monte Carlo và Q-learning trong 3 phase khác nhau

2.3.3.1. Phase 1

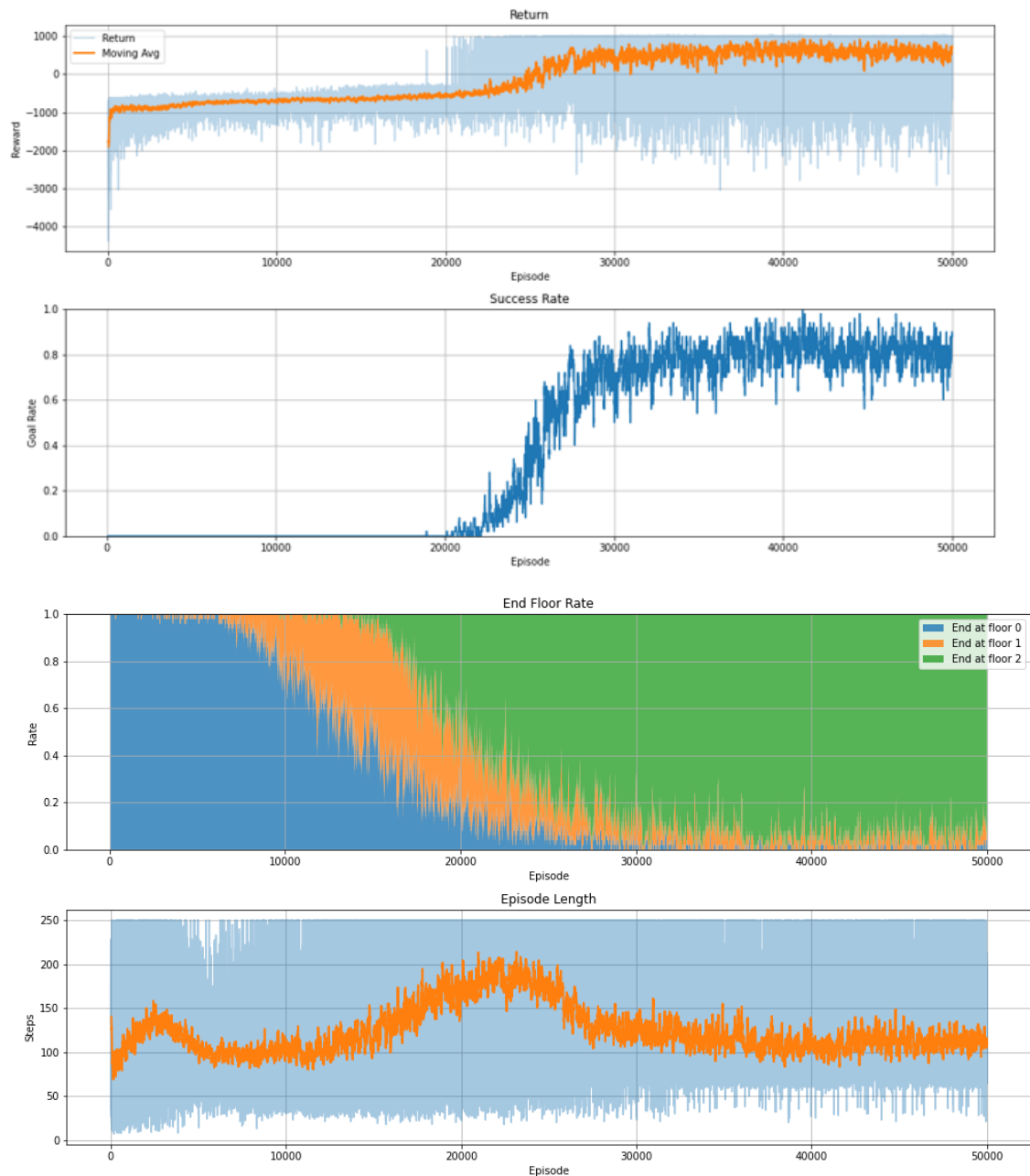
Map cố định, seed 42, soft wall = 0, số lượng bomb tối đa = 0.

Monte Carlo:



Hình 17: Đánh giá Phase 1 Monte Carlo

Q- Learning:



Hình 18: Đánh giá Phase 1 Q-Learning

Kết quả thực nghiệm cho thấy sự khác biệt rõ rệt về hiệu suất giữa hai thuật toán. Q-Learning đạt tỷ lệ thành công 80 - 85% ở giai đoạn cuối, với phần thưởng trung bình dương (+437 đến +647) và tổng cộng 19.403 lần hoàn thành mục tiêu. Ngược lại, Monte Carlo Control không đạt được goal nào sau 50.000 episodes, phần thưởng trung bình luôn âm sâu (-879 đến -728), và success rate bằng 0, dù có khả năng lên tầng cuối từ khoảng episode 38.000.

Về hội tụ, Q-Learning thể hiện rõ rệt và sớm hơn: bắt đầu có goal từ ~20.000 episodes, tăng nhanh từ 22.000 episodes và ổn định quanh 80% từ episode 38.000. Phần

thường trung bình chuyển sang dương tại khoảng episode 28.000, cho thấy chính sách đã hiệu quả. Trong khi đó, Monte Carlo không có dấu hiệu hội tụ; dù phần thưởng cải thiện nhẹ, nhưng không dẫn đến hành vi đạt mục tiêu.

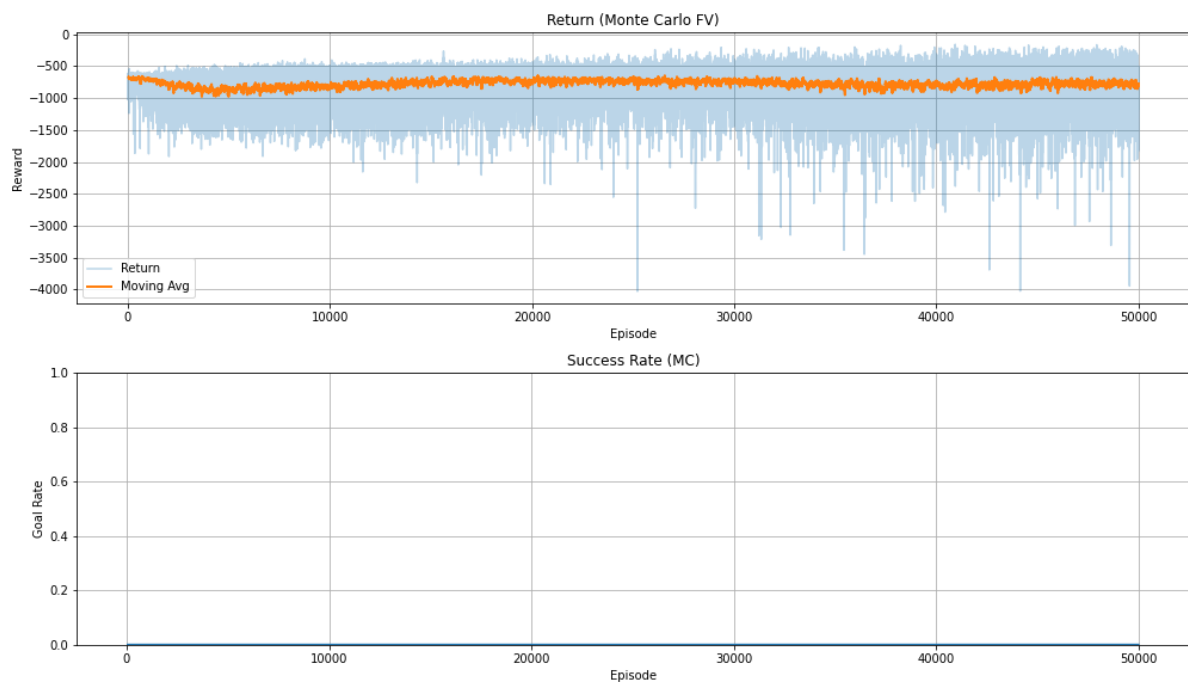
Xét cấu trúc kết thúc episode, Monte Carlo có số lần stagnation rất cao (36.271/50.000) và timeout tăng dần, cho thấy xu hướng kéo dài episode nhưng không tiến tới mục tiêu - biểu hiện của chính sách thụ động. Ngược lại, Q-Learning giảm dần death và tăng goal theo thời gian, phản ánh sự cân bằng tốt giữa khai thác và khám phá.

Về kích thước bảng tra cứu, Q-Learning đạt 168.070 cặp (s, a), lớn hơn Monte Carlo (101.209), cho thấy khả năng khám phá trạng thái tốt hơn nhờ cập nhật TD.

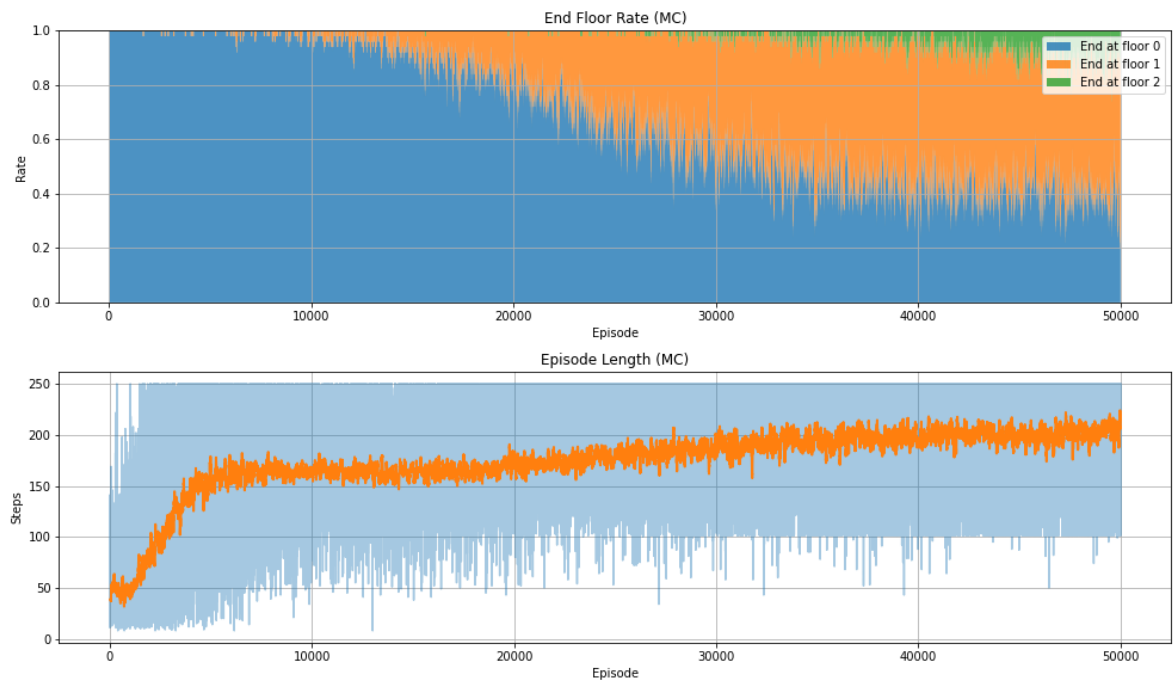
2.3.3.2. Phase 2

Map cố định, seed 42, soft wall = 5, số lượng bomb tối đa = 2

Monte Carlo:

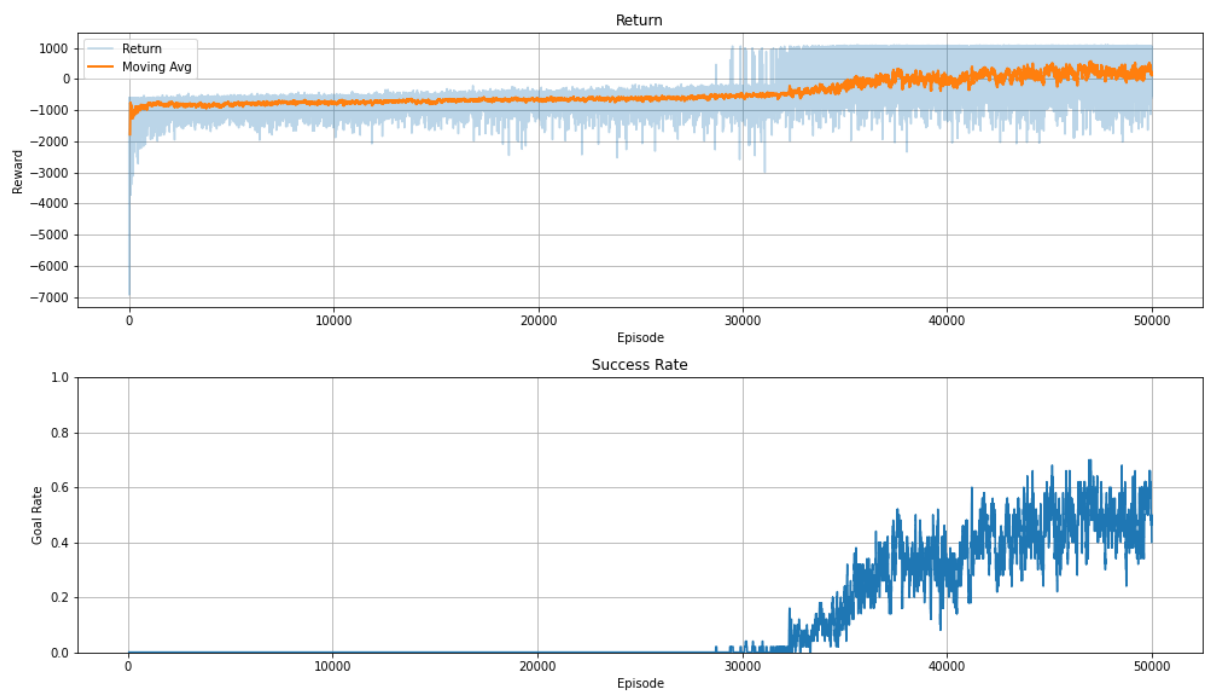


Hình 19.1: Đánh giá Phase 2 Monte Carlo

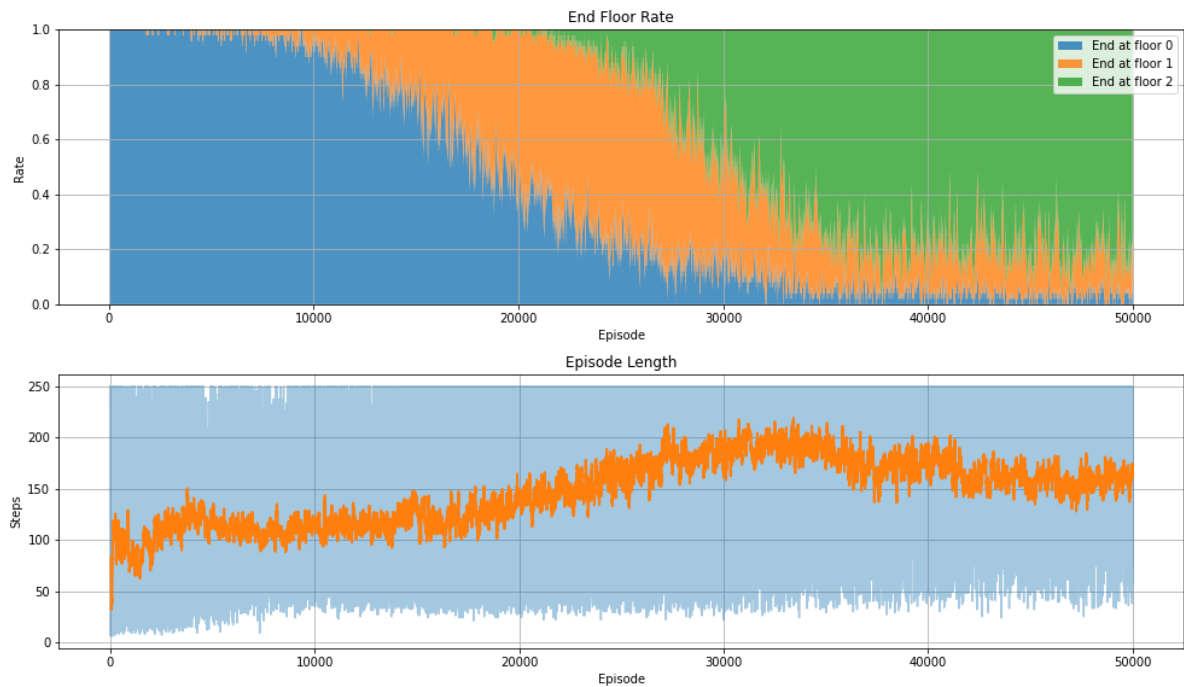


Hình 19.2: Đánh giá Phase 2 Monte Carlo

Q-Learning:



Hình 20.1: Đánh giá Phase 2 Q-Learning



Hình 20.2: Đánh giá Phase 2 Q-Learning

Tương tự Phase 1, Phase 2 tiếp tục cho thấy sự chênh lệch rõ rệt giữa hai thuật toán, đồng thời phản ánh mức độ khó cao hơn của môi trường. Monte Carlo Control vẫn không đạt được goal nào sau 50.000 episodes, với phần thưởng trung bình khoảng -777.97, chỉ cải thiện nhẹ so với Phase 1. Ngược lại, Q-Learning đạt 6.149 goal, phần thưởng trung bình +204.51 và success rate khoảng 48–50%. Dù thấp hơn Phase 1 (~80%), kết quả này chủ yếu do môi trường phức tạp hơn.

Về hội tụ, Q-Learning chậm hơn đáng kể: bắt đầu có goal từ khoảng episode 30.000, success rate tăng dần và ổn định ở mức ~50% vào cuối quá trình; phần thưởng trung bình chuyển sang dương tại khoảng episode 42.000. Điều này cho thấy môi trường có bom và tường mềm đòi hỏi thời gian khám phá dài hơn. Ngược lại, Monte Carlo không có dấu hiệu hội tụ; phần thưởng dao động quanh mức -735 đến -814, cho thấy sự thiếu ổn định.

Về khả năng kế thừa, Q-Learning tận dụng tốt mô hình từ Phase 1: dù môi trường thay đổi, các giá trị Q sẵn có vẫn hỗ trợ quá trình học, giúp duy trì xu hướng hội tụ. Trong khi đó, Monte Carlo hầu như không hưởng lợi từ việc khởi tạo; dù phần thưởng ban đầu có vẻ tốt hơn, hiệu suất nhanh chóng suy giảm và không ghi nhận goal nào, do cơ chế cập nhật cuối episode thiếu tín hiệu phần thưởng dương.

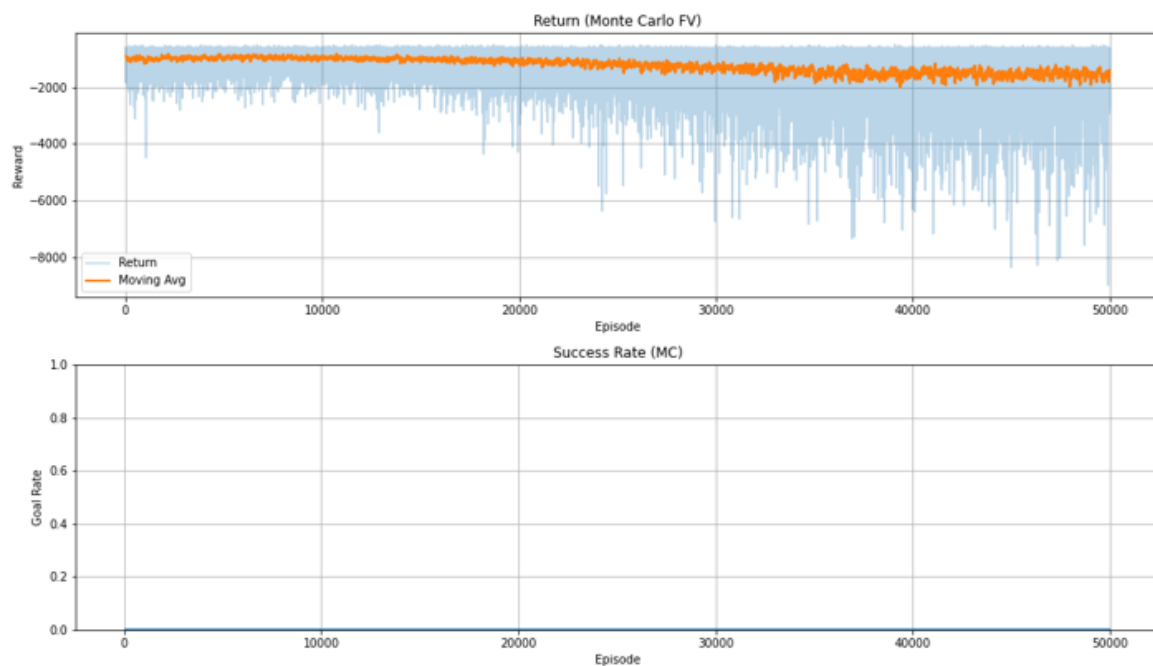
Xét cấu trúc kết thúc episode, Monte Carlo tiếp tục thể hiện chính sách thụ động: stagnation chiếm 33.766 episodes, timeout cao, và death thấp, cho thấy agent tránh rủi ro nhưng không tiến tới mục tiêu. Ngược lại, Q-Learning có số death tăng (29.753) do môi trường nguy hiểm hơn, nhưng vẫn duy trì tăng trưởng goal ổn định, phản ánh quá trình khám phá hiệu quả.

Về không gian trạng thái, Q-Learning mở rộng Q-table lên 455.996 cặp (s,a) (~2.7 lần Phase 1), trong khi Monte Carlo đạt 259.706 cặp. Tuy nhiên, với Monte Carlo, sự gia tăng này không chuyển hóa thành hiệu suất, cho thấy hạn chế cốt lõi nằm ở cơ chế cập nhật chưa phù hợp với bài toán phần thưởng thưa.

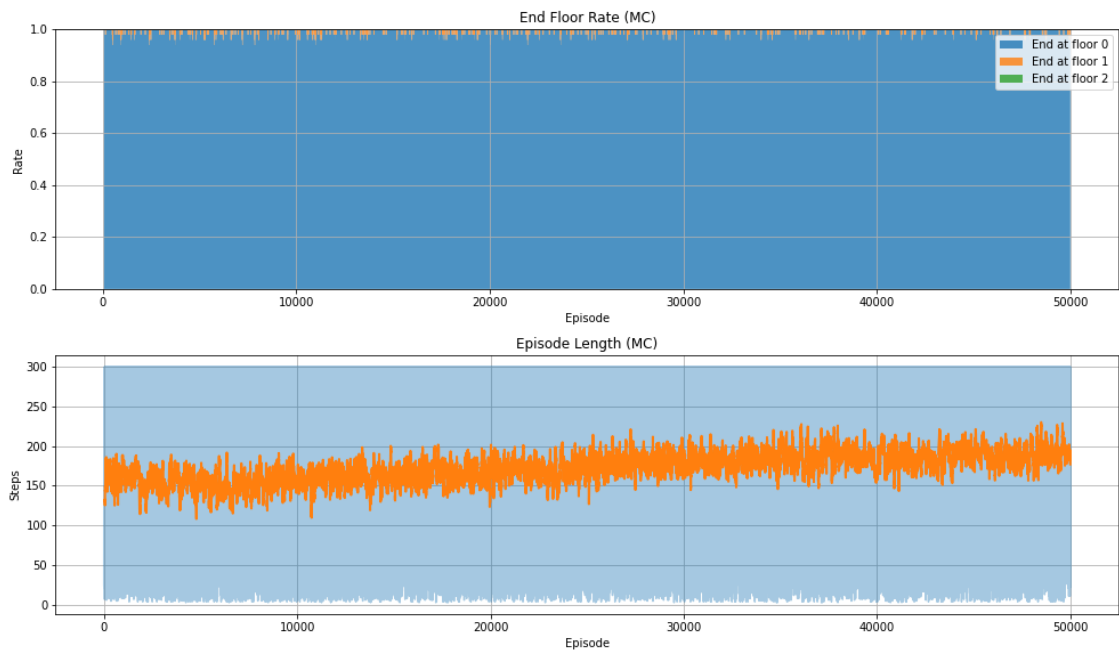
2.3.3.3. Phase 3

Map random, seed 42, soft wall = 5, số lượng bomb tối đa = 2

Monte Carlo:

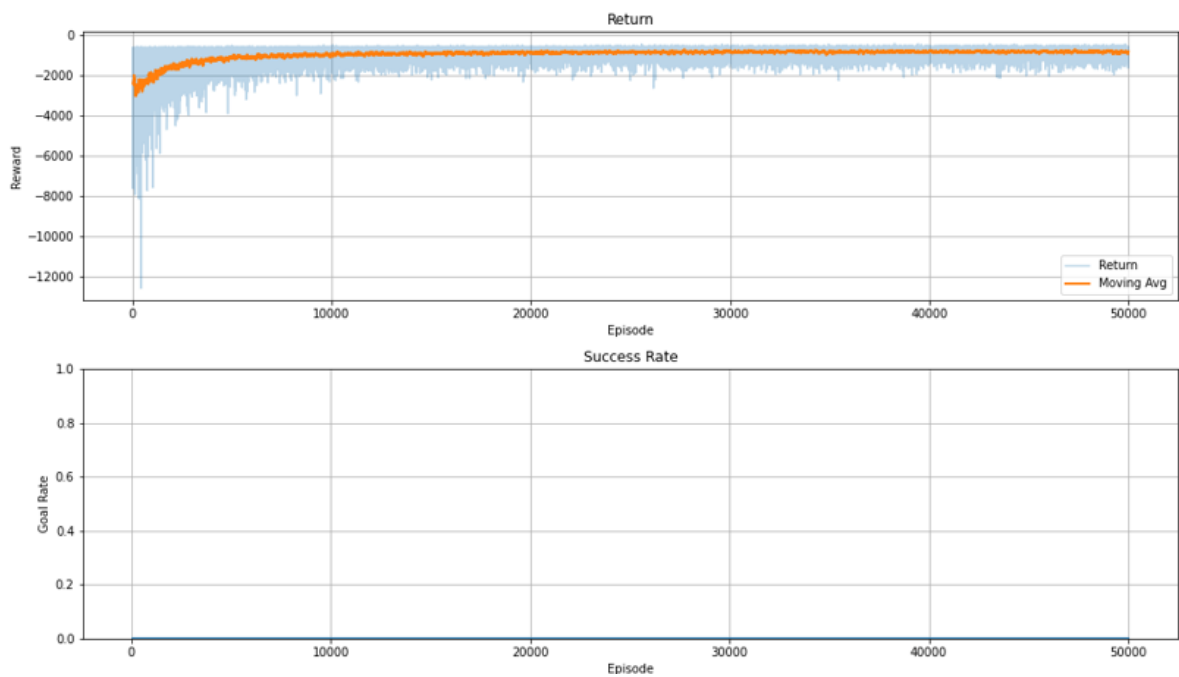


Hình 21.1: Đánh giá Phase 3 Monte Carlo

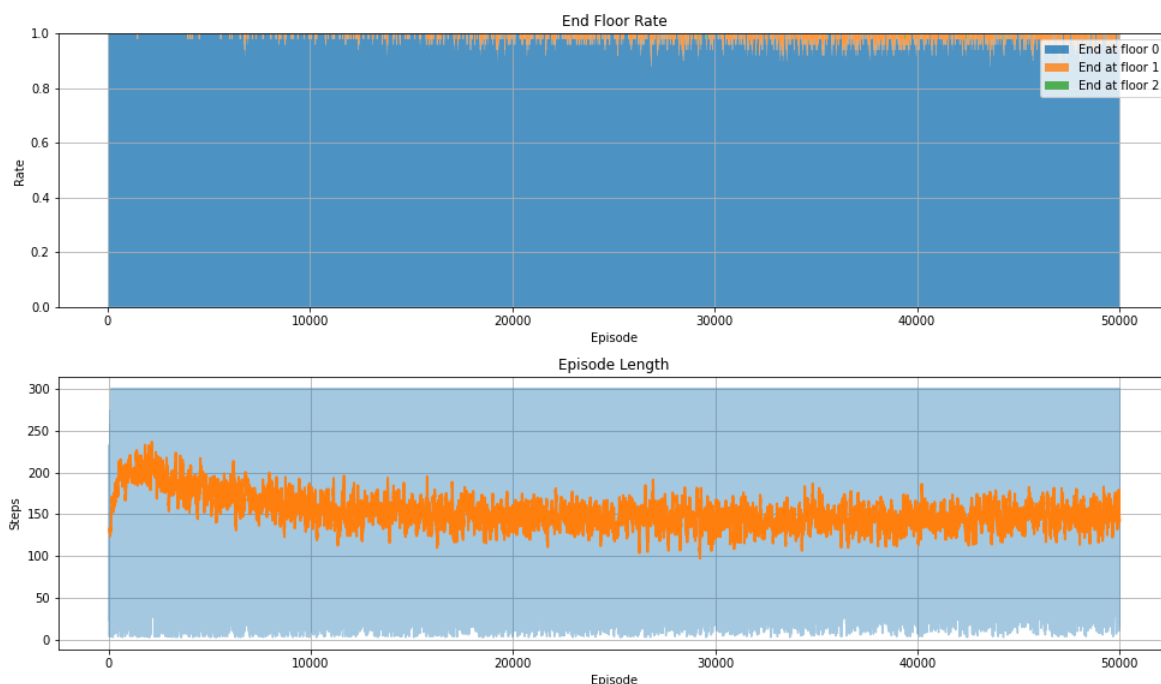


Hình 21.2: Đánh giá Phase 3 Monte Carlo

Q-Learning:



Hình 22.1: Đánh giá Phase 3 Q-Learning



Hình 22.2: Đánh giá Phase 3 Q-Learning

Phase 3 là giai đoạn duy nhất mà cả hai thuật toán đều thất bại hoàn toàn, với success rate = 0 và không có goal nào sau 50.000 episodes. Q-Learning đạt phần thưởng trung bình -843.90, trong khi Monte Carlo kém hơn đáng kể với -1.568.32. Đây là sự sụp đổ rõ rệt, đặc biệt khi Q-Learning từng đạt ~80% (Phase 1) và ~48% (Phase 2).

Nguyên nhân chính đến từ tính ngẫu nhiên của môi trường (Random = True), khiến không gian trạng thái trở nên gần như vô hạn. Với đặc trưng của tabular RL, mỗi cấu hình bản đồ tạo ra trạng thái mới, dẫn đến hiện tượng bùng nổ không gian trạng thái và không thể tái sử dụng kinh nghiệm. Điều này thể hiện qua kích thước bảng tăng đột biến: Q-table đạt 3.907.127 cặp (s,a) (~8.6 lần Phase 2), MC Table đạt 3.345.735 (~12.9 lần), với tốc độ tăng trưởng liên tục và không có dấu hiệu bão hòa.

Dù đều thất bại, Q-Learning vẫn ổn định hơn: phần thưởng duy trì quanh -834 đến -851 ở nửa sau huấn luyện. Ngược lại, Monte Carlo suy giảm rõ rệt theo thời gian (-1.009 → -1.568), cho thấy mất ổn định. Khi epsilon giảm, Monte Carlo khai thác chính sách đã học nhưng chính sách này không còn phù hợp với các bản đồ mới, dẫn đến hiệu suất ngày càng kém. Q-Learning, nhờ cập nhật TD, thích nghi tốt hơn với trạng thái mới dù không hội tụ.

Về cấu trúc kết thúc episode, Q-Learning có tỷ lệ death rất cao (37.050; 74.1%), phản ánh việc agent chủ động khám phá nhưng liên tục thất bại trong môi trường nguy

hiêm. Ngược lại, Monte Carlo có tỷ lệ stagnation cao hơn (14.599; 29.2%), tiếp tục thể hiện chính sách thụ động. Cả hai gần như không vượt qua được tầng đầu của môi trường.

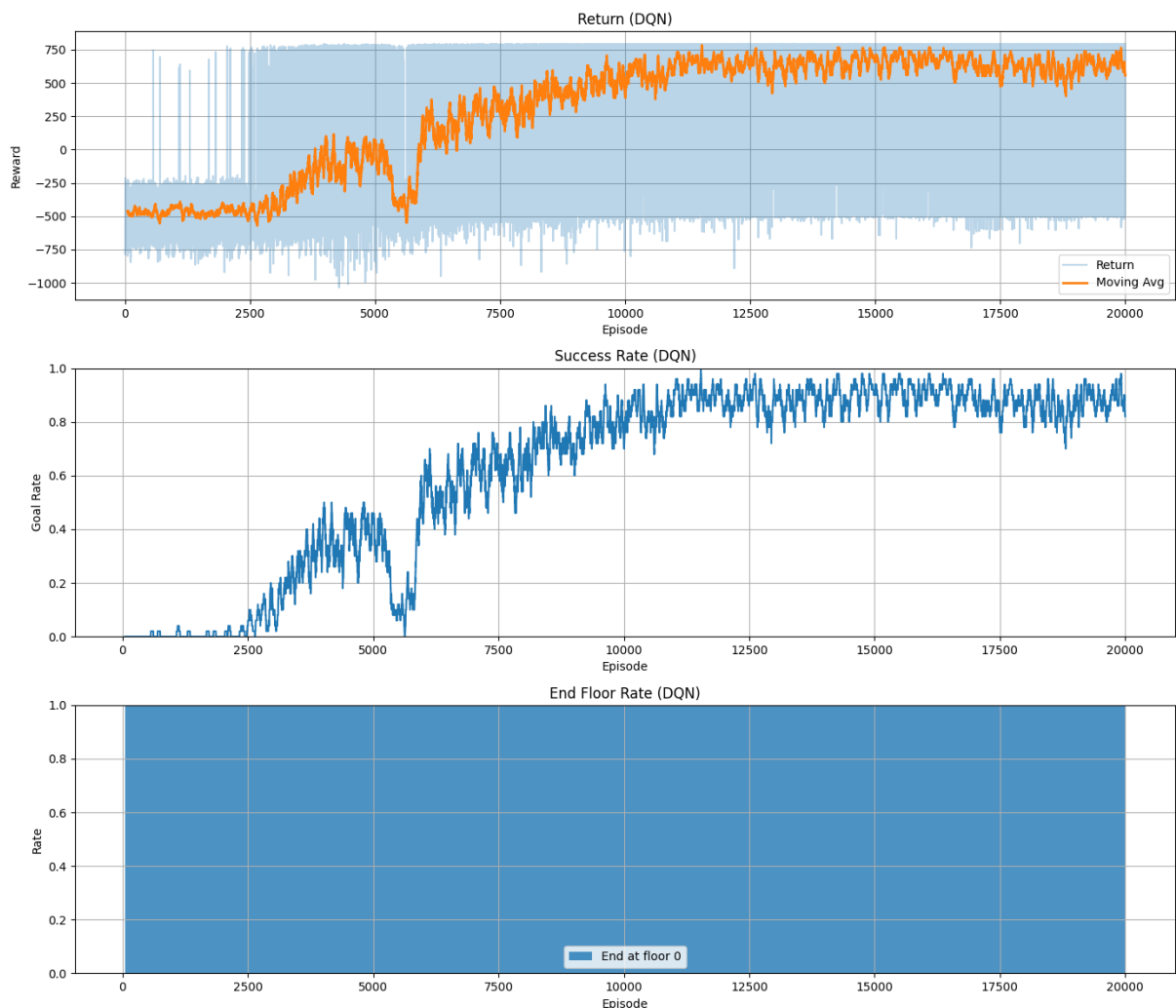
Cuối cùng, kích thước bảng giá trị trở nên không khả thi trong thực tế (Q-Learning $\sim 3.9M$, Monte Carlo $\sim 3.3M$ cặp), với mức tăng ~ 150.000 cặp mỗi 2.000 episodes và không có dấu hiệu hội tụ. Điều này cho thấy tabular RL không phù hợp với môi trường có không gian trạng thái thay đổi liên tục.

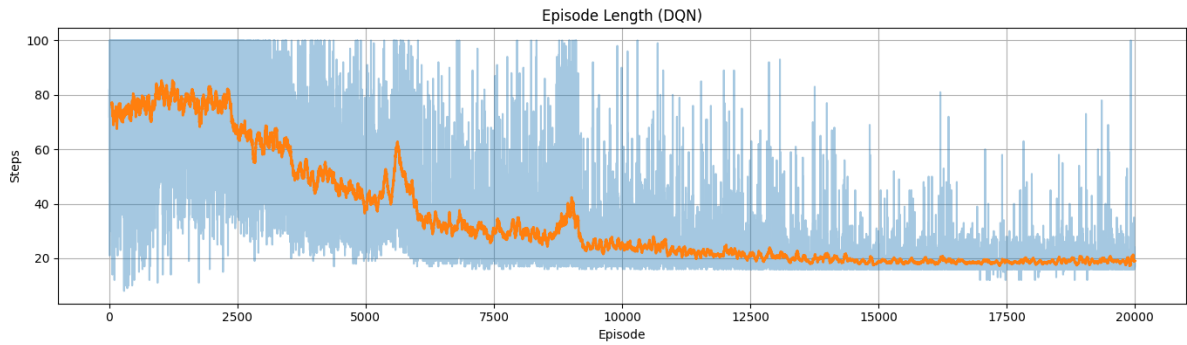
2.3.4. Đánh giá Double DQN

Đánh giá Double DQN thông qua 4 Phase khác nhau. Mỗi phase được train 20k episode.

2.3.4.1. Phase 1

Map cố định, Seed 42, số lượng soft wall = 5, số lượng bomb tối đa = 2, tổng số lượng tầng = 1, số bước tối đa – maxSteps = 100.





Hình 24: Đánh giá Phase 1 Double DQN

Double DQN cho thấy khả năng hội tụ nhanh và rõ ràng. Các goal đầu tiên xuất hiện từ episode 800, và tỷ lệ thành công tăng mạnh từ khoảng episode 3.200 (từ 8% lên 28% chỉ sau 800 episodes). Phần thưởng trung bình chuyển từ âm sang dương tại khoảng episode 7.200, sau đó tiếp tục tăng ổn định, đạt +672.34 ở episode 16.000.

Kết thúc huấn luyện (episode 20.000), mô hình đạt tỷ lệ thành công 89%, phần thưởng trung bình +640.94 và điểm trung bình toàn bộ là +291.72, với tổng 12.399 goal. Số bước trung bình giảm từ ~75 xuống ~19, cho thấy agent không chỉ đạt mục tiêu mà còn tối ưu hóa đường đi hiệu quả hơn.

Số lần death tăng trong giai đoạn đầu do khám phá (299 → 5.345), nhưng giảm rõ rệt sau episode 12.000 (chỉ còn ~67–110 mỗi 800 episodes). Đồng thời, goal duy trì ổn định (~700–720/800 ep), trong khi timeout gần như bằng 0 và stagnation không đáng kể. Điều này cho thấy agent đã học được chính sách ổn định, chủ động và hiệu quả

2.3.4.2. Phase 2

Map cố định, Seed 42, số lượng soft wall = 5, số lượng bomb tối đa = 2, tổng số lượng tầng = 2, số bước tối đa – maxSteps = 200.



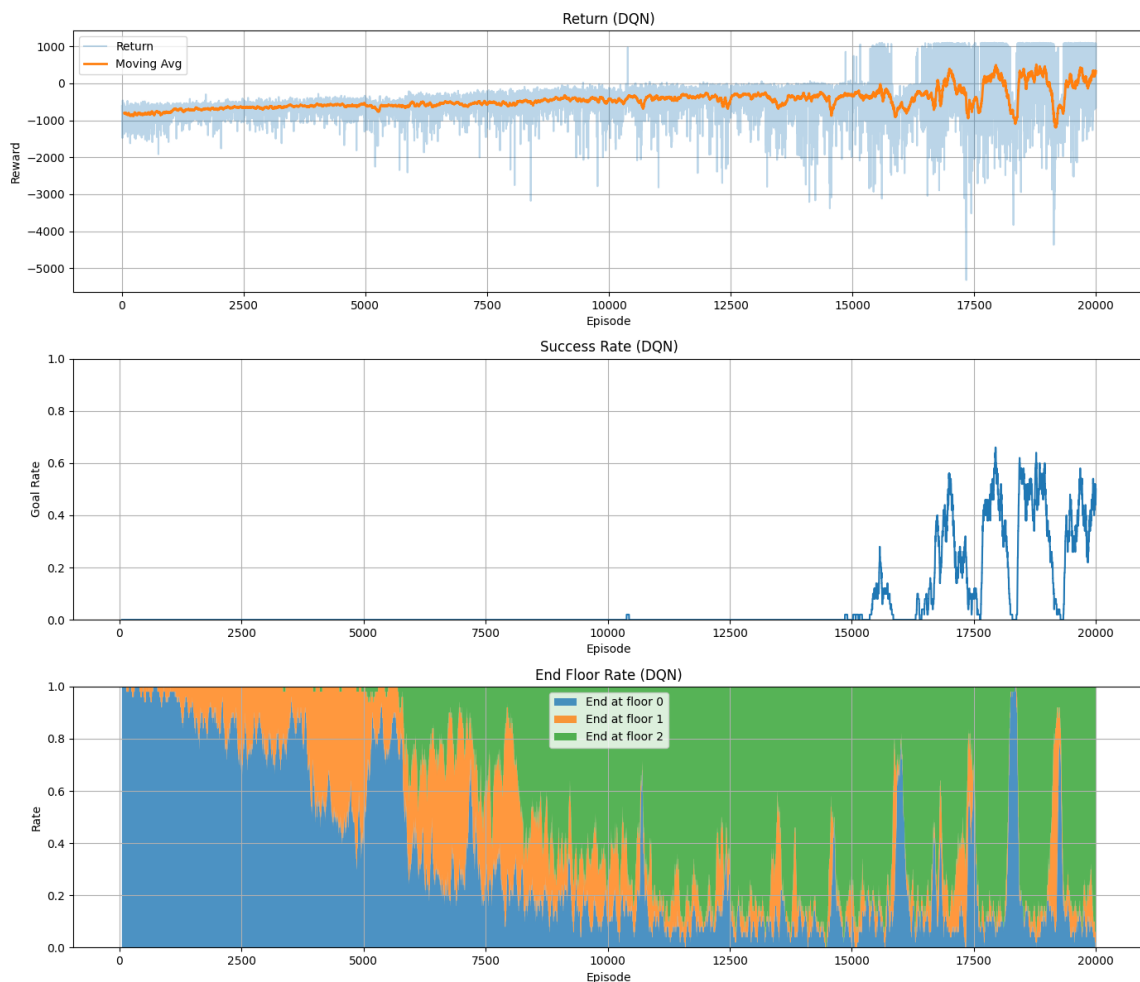
Hình 25: Đánh giá Phase 2 Double DQN

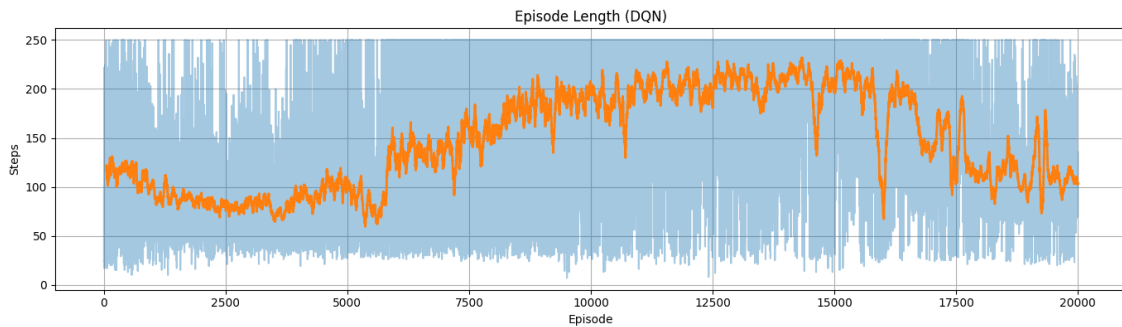
So với Phase 1, Double DQN hội tụ chậm hơn đáng kể trong Phase 2. Trong 8.000 episodes đầu, tỷ lệ thành công duy trì ở mức 0%. Goal đầu tiên xuất hiện tại episode 8.800, và bước nhảy vọt xảy ra ở episode 9.600 khi success_rate tăng lên và avg_reward chuyển dương, cho thấy quá trình học mang tính phi tuyến với giai đoạn tích lũy trước khi đột phá.

Sau khi hội tụ, hiệu suất tiếp tục cải thiện và đạt đỉnh tại episode 19.200 với $\text{success_rate} = 90\%$ và $\text{avg_reward} = +783.28$. Kết thúc huấn luyện tại episode 20.000, mô hình duy trì $\text{success_rate} = 87\%$ và $\text{avg_reward} = +744.54$, với tổng 7.985 goals trên 20.000 episodes. Số bước trung bình giảm từ ~107 bước (giai đoạn đầu) xuống còn ~44 – 50 bước ở giai đoạn cuối. Đáng chú ý, phần thưởng trung bình cuối phase (+744.54) cao hơn Phase 1 (+640.94), cho thấy môi trường 2 tầng mang lại phần thưởng tích lũy lớn hơn khi agent hoàn thành mục tiêu.

2.3.4.3. Phase 3

Map cố định, Seed 42, số lượng soft wall = 5, số lượng bomb tối đa = 2, tổng số lượng tầng = 3, số bước tối đa – $\text{maxSteps} = 250$.





Hình 26: Đánh giá Phase 3 Double DQN

Phase 3 cho thấy Double DQN không hội tụ rõ ràng. Trong 10.400 episodes đầu, agent hầu như không đạt goal, với `success_rate` duy trì 0%. Phần thưởng trung bình cải thiện chậm từ -821.10 lên -363.16 nhưng không ổn định ở mức dương; điểm trung bình toàn phase là -451.31. Dấu hiệu học xuất hiện muộn và yếu: `success_rate` đạt 1% tại ep 15.200, cao nhất 41% tại ep 19.200 rồi giảm còn 33% ở cuối, cho thấy mô hình chưa hội tụ sau 20.000 episodes.

Kết thúc huấn luyện, agent đạt 1.142 goals, thấp hơn nhiều so với Phase 1 (12.399) và Phase 2 (7.985). `Success_rate` cuối là 33% với `avg_reward` = -19.16, phản ánh khả năng khai thác môi trường 3 tầng còn hạn chế. Số bước trung bình giảm từ ~200 xuống ~110, nhưng chưa đủ để thể hiện một chính sách tối ưu.

Dữ liệu vị trí kết thúc cho thấy agent dần lên được tầng 2 (chiếm đa số từ ep 9.600), nhưng chủ yếu dừng lại mà không hoàn thành mục tiêu. Đồng thời, `timeout` (4.582) và `stagnation` (3.209) ở mức rất cao, cho thấy agent thường xuyên rơi vào trạng thái bế tắc, không thể tìm ra hành động có ích trong môi trường phức tạp hơn.

2.3.4.4. Phase 4

Map random, Seed 42, số lượng soft wall = 5, số lượng bomb tối đa = 2, tổng số lượng tầng = 3, số bước tối đa – `maxSteps` = 250.



Hình 27: Đánh giá Phase 4 Double DQN

Phase 4 là giai đoạn thất bại nghiêm trọng của Double DQN. Với môi trường 3 tầng và bản đồ ngẫu nhiên, mô hình gần như không học được gì trong 20.000 episodes: chỉ đạt 2 goals, success_rate duy trì 0%, và điểm trung bình toàn phase là -1.049.46 thấp hơn nhiều so với Phase 3.

Không giống Phase 3 (vẫn có cải thiện nhẹ), phần thưởng ở Phase 4 giảm dần theo thời gian, từ -842.87 xuống khoảng -1.100 đến -1.240 ở nửa sau huấn luyện. Khi epsilon đạt mức tối thiểu (ep 16.000), hiệu suất vẫn không cải thiện, cho thấy chính sách học được không có giá trị khai thác.

Các chỉ số kết thúc episode cho thấy agent mất hoàn toàn định hướng: timeout (6.977) vượt death (6.313) và stagnation lên tới 6.708 - cao nhất trong tất cả các phase. Đồng thời, agent gần như bị kẹt ở tầng 0 (670–790/800 episodes), trái ngược với Phase 3 khi vẫn có khả năng lên tầng 2.

Nguyên nhân chính là sự kết hợp giữa sparse reward và môi trường thay đổi liên tục, khiến mỗi episode gần như độc lập. Điều này ngăn mạng học được biểu diễn tổng quát, dẫn đến thất bại trong việc học chính sách điều hướng.

CHƯƠNG 3: KẾT LUẬN

3.1. Kết quả đạt được

Đồ án đã triển khai bài toán học tăng cường trong môi trường mê cung đa tầng (Multi-Floor Maze) với thiết kế cố định và ngẫu nhiên hóa bản đồ, cơ chế thực thể động và không gian trạng thái quan sát được gồm 19 chiều. Ba thuật toán đã được cài đặt, bao gồm: First-Visit Monte Carlo Control, Q-Learning và Double Deep Q-Network.

Về kết quả thực nghiệm, Q-Learning thể hiện hiệu suất vượt trội so với Monte Carlo Control trong điều kiện bản đồ cố định. Cụ thể, trong Phase 1, Q-Learning đạt tỷ lệ thành công 80–85% và phần thưởng trung bình từ +437 đến +647 sau 50.000 episodes, trong khi Monte Carlo Control không đạt được bất kỳ goal nào, với phần thưởng trung bình duy trì âm sâu và tỷ lệ stagnation chiếm hơn 72% tổng số episodes. Sự chênh lệch này xuất phát từ bản chất cơ chế cập nhật: Q-Learning cập nhật trực tuyến theo từng bước thông qua ước lượng TD, cho phép lan truyền tín hiệu phần thưởng sớm và liên tục; trong khi Monte Carlo chỉ cập nhật sau khi một episode kết thúc hoàn toàn, khiến thuật toán gặp khó khăn nghiêm trọng trong môi trường có phần thưởng thưa thớt.

Tuy nhiên, cả hai thuật toán tabular đều thất bại hoàn toàn trong Phase 3 khi bản đồ được ngẫu nhiên hóa mỗi episode. Kích thước bảng Q-table bùng nổ lên hơn 3,9 triệu cặp (s, a) mà không có dấu hiệu bão hòa, phản ánh giới hạn cố hữu của phương pháp bảng tra cứu khi đối mặt với không gian trạng thái thay đổi liên tục. Điều này xác nhận rằng các phương pháp tabular RL không thể khái quát hóa sang các cấu hình môi trường chưa từng gặp trong quá trình huấn luyện.

Double DQN, với kiến trúc mạng nơ-ron đa nhánh xử lý ba nhóm đặc trưng (cục bộ, tương tác, ngữ cảnh) cùng các cơ chế Experience Replay và Target Network với soft update, đã chứng minh khả năng học hiệu quả trong môi trường có kích thước trạng thái lớn và bản đồ cố định. Trong Phase 1 và Phase 2 (1 tầng và 2 tầng, bản đồ cố định), mô hình đạt tỷ lệ thành công lần lượt là 89% và 87% chỉ sau 20.000 episodes, với số bước trung bình giảm đáng kể, phản ánh chính sách được tối ưu hóa theo hướng cả hiệu quả lẫn tốc độ. Đặc biệt, phần thưởng trung bình ở Phase 2 (+744.54) vượt Phase 1 (+640.94), cho thấy mô hình có khả năng khai thác phần thưởng tích lũy từ môi trường nhiều tầng.

Dù vậy, Double DQN vẫn gặp khó khăn khi cả độ phức tạp môi trường lẫn tính ngẫu nhiên bản đồ tăng. Trong Phase 3 (3 tầng, bản đồ cố định), mô hình chỉ đạt tỷ lệ

thành công tối đa 41% và không ổn định; và trong Phase 4 (3 tầng, bản đồ ngẫu nhiên), mô hình thực tế không học được gì sau 20.000 episodes với điểm trung bình âm 1.049.46.

Nhìn tổng thể, quá trình đánh giá cho thấy một xu hướng. Q-Learning là lựa chọn phù hợp cho môi trường có không gian trạng thái hữu hạn và bản đồ cố định nhờ cơ chế cập nhật TD nhanh; Double DQN vượt trội hơn khi môi trường đủ phức tạp để các phương pháp tabular không khả thi, nhưng vẫn cần sự ổn định của bản đồ để hội tụ; còn Monte Carlo Control, thì kém hiệu quả nhất trong tất cả các kịch bản do cơ chế cập nhật cuối episode không tương thích với bài toán phần thưởng thưa và môi trường nhiều bước.

3.2. Hạn chế và hướng phát triển

Bên cạnh những kết quả đạt được, đồ án vẫn tồn tại một số hạn chế đáng chú ý. Thứ nhất, các phương pháp tabular (Q-Learning và Monte Carlo) hoàn toàn mất khả năng tổng quát hóa khi bản đồ được ngẫu nhiên hóa do bùng nổ không gian trạng thái. Thứ hai, Double DQN sử dụng kiến trúc MLP đa nhánh phẳng vẫn chưa đủ năng lực biểu diễn để xử lý cấu trúc không gian của môi trường mê cung khi cả số tầng lẫn tính ngẫu nhiên tăng đồng thời, dẫn đến thất bại trong Phase 4. Thứ ba, vector trạng thái 19 chiều, dù được thiết kế cô đọng, vẫn chưa mã hóa được đầy đủ thông tin toàn cục của bản đồ. Cuối cùng, số lượng episodes huấn luyện còn hạn chế (20.000 cho Double DQN và 50.000 cho các phương pháp tabular) có thể chưa đủ để các thuật toán hội tụ hoàn toàn trong các kịch bản phức tạp.

Từ những hạn chế trên, có thể đề xuất một số hướng phát triển tiềm năng cho các nghiên cứu tiếp theo. Về kiến trúc mô hình, việc thay thế MLP bằng mạng tích chập (CNN) hoặc mạng chú ý (Attention/Transformer) để xử lý trực tiếp bản đồ dạng lưới 2D có thể cải thiện đáng kể khả năng biểu diễn đặc trưng không gian. Về thuật toán, các phương pháp tiên tiến hơn như Proximal Policy Optimization (PPO) hay Asynchronous Advantage Actor-Critic (A3C) có thể được khảo sát nhằm tăng tính ổn định trong huấn luyện với môi trường bất định cao. Ngoài ra, kỹ thuật học có phân cấp (Hierarchical RL) có thể giúp chia nhỏ bài toán tìm đường đa tầng thành các mục tiêu con tuần tự, phù hợp hơn với cấu trúc của bài toán. Cuối cùng, việc áp dụng các phương pháp học chuyển tiếp (Transfer Learning) từ bản đồ đơn giản sang phức tạp, hoặc kỹ thuật curriculum learning để điều chỉnh độ khó môi trường theo tiến trình học, cũng là những hướng nghiên cứu để giải quyết bài toán khái quát hóa trong môi trường ngẫu nhiên hóa.